Лабораторный практикум по курсу
« Структуры и алгоритмы обработки данных »

Оглавление

1. Лабораторная работа - Методы сортировки 5
1.1 Теоретические сведения 5
1.2 Сортировка выбором 6
1.3 Сортировка вставкой 6
1.4 Сортировка Шелла 10
1.5 Быстрая сортировка (сортировка Хоара) 11
1.6 Сортировка в нелинейных структурах 12
1.6.1 Турнирная сортировка 12
1.6.2 Пирамидальная сортировка 12
1.7 Функция сложности алгоритма 13
1.8 Методика выполнения лабораторной работы 14
1.9 Пояснения к выполнению работы. 16
2 Лабораторная работа -Методы поиска 18
2.1 Теоретические сведения 18
2.2 Последовательный поиск. 18
2.3 Бинарный поиск. 18
2.4 Фибоначчиев поиск. 19
2.5 Интерполяционный поиск. 20
2.6 Поиск по бинарному дереву. 21
2.7 Поиск по сбалансированному дереву. 22
2.8 Поиск по бору 23
2.9 Поиск хешированием 24
2.10 Алгоритмы поиска словесной информации 25
2.10.1 Алгоритм Кнута - Морриса - Пратта 25
2.10.2 Алгоритм Бойера - Мура 26
2.10.3 Алгоритм Рабина 27
2.11 Методика выполнения лабораторной работы 28
3 Лабораторная работа -Итеративные и рекурсивные алгоритмы 30
3.1 Теоретические сведения 30
3.2 Итеративный алгоритм. 30
3.2.1 Итеративное вычисление факториала 30
3.2.2 Рекурсивное вычисление факториала 31
3.3 Рекурсивные структуры данных 32
3.3.1 Формирование бинарного дерева 36
3.3.2 Рекурсивная процедура обхода узлов дерева сверху-вниз 37
3.4 Лабораторное задание 37
3.5 Требования к отчету 38
3.6 Контрольные вопросы 38
3.7 Литература 38
4 Лабораторная работа - Алгоритмы построения остовного дерева сети 39
4.1 Теоретические сведения 39
4.2 Алгоритмы Крускала и Прима 40
4.2.1 Пример со схемой микрорайона 42
4.2.2 Пример со схемой компьютерной сети 44
4.3 Лабораторное задание 45
4.4 Требования к отчёту 45
4.5 Литература 45
4.6 Задание к лабораторной работе 4 46
4.7 Решение задач 50
5 Лабораторная работа - Алгоритмы нахождения на графах кратчайших путей 52

5.1 Теоретические сведения 52
5.2 Метод динамического программирования. 52
5.3 Пример определения кратчайшего пути №1 53
5.4 Пример нахождения кратчайшего пути при условии, что граф неориентированный№2 54
5.5 Метод Дейкстры 56
5.6 Алгоритм Флойда 58
5.7 Алгоритм Йена 60
5.8 Алгоритм Беллмана- Форда 61
5.9 Литература 61
5.10 Лабораторное задание 62
5.11 Требования к отчету 63
5.12 Варианты заданий 64
5.12.1 Задание 1: Найти кратчайший путь на графе между тремя парами вершин методом динамического программирования 64
5.12.2 Задание 2: Найти кратчайший путь между тремя парами вершин методом Дейкстры 68
5.13 Решение задач 73
5.14 Задание на разработку программы 75
6 Лабораторная работа -Эвристические алгоритмы 80
6.1 Теоретические сведения 80
6.2 Волновой алгоритм 80
6.3 Двухлучевой алгоритм. 81
6.4 Четырехлучевой алгоритм 82
6.5 Маршрутный алгоритм. 82
6.6 Геометрическая модель задачи о лабиринте 84
6.7 Алгоритмы составления расписания. 86
6.8 Литература 88
6.9 Лабораторное задание. 88
6.10 Требования к отчету 88
6.11 Решение задач 89

В лабораторном практикуме рассмотрен широкий круг алгоритмов обработки линейных и нелинейных структур данных, без знания которых невозможно современное компьютерное моделирование. Приведены основные понятия и определения, технология работы и фрагменты программ. В конце каждой лабораторной работы имеются варианты заданий и задачи для самостоятельного решения. Практикум предназначен для студентов, аспирантов и преподавателей.

1. Лабораторная работа - Методы сортировки

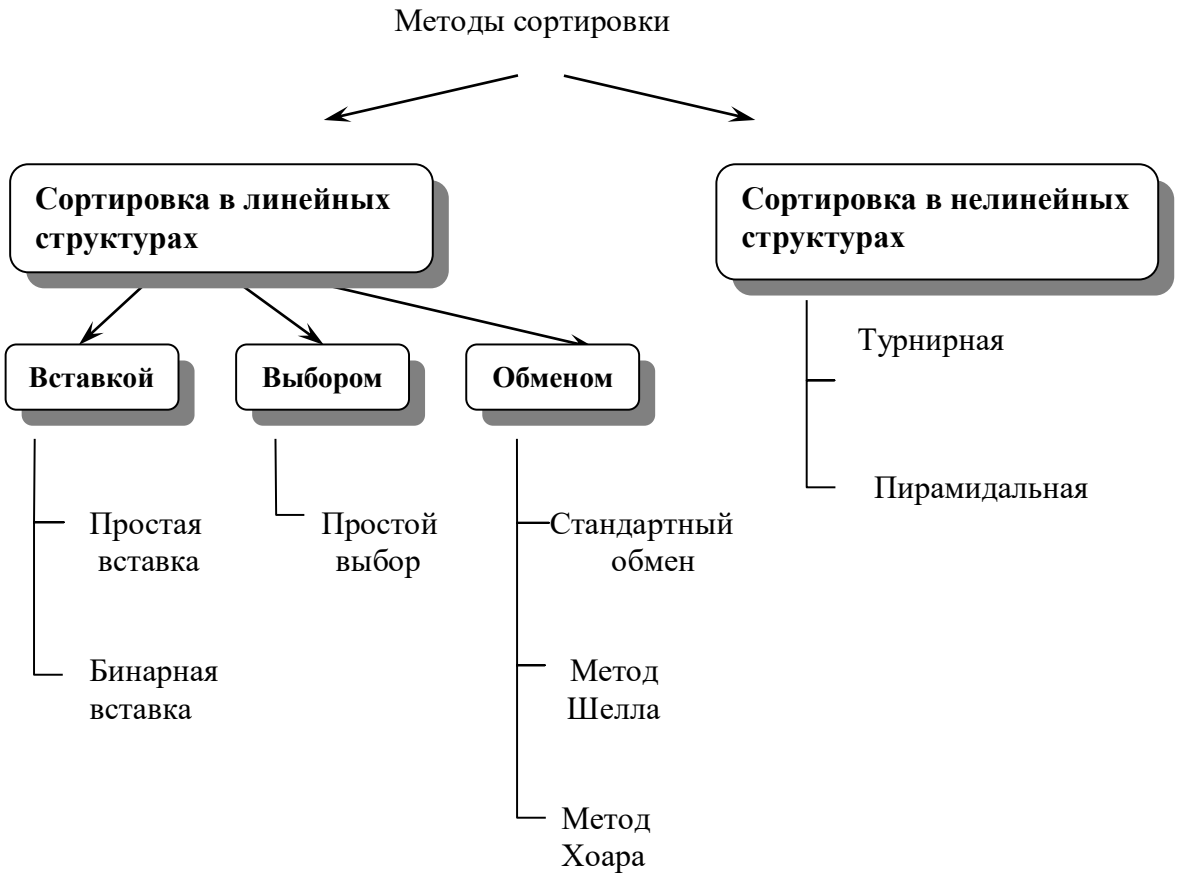
Цель работы: ознакомление с алгоритмами сортировки линейных и нелинейных структур и методикой оценки эффективности алгоритмов.
Продолжительность работы: - 2 часа.

1 Теоретические сведения

Упорядочение элементов множества в возрастающем или убывающем порядке называется сортировкой.

С упорядоченными элементами проще работать, чем с произвольно расположенными: легче найти необходимые элементы, исключить ,вставить новые. Сортировка применяется при трансляции программ, при организации наборов данных на внешних носителях, при создании библиотек, каталогов, баз данных и т.д.

Алгоритмы сортировки можно разбить на следующие группы:



Обычно сортируемые элементы множества называют записями и обозначают через

$\mathbf{k_1, k_2, \ldots, k_n}$.

2 Сортировка выбором

Сортировка выбором состоит в том, что сначала в неупорядоченном списке выбирается и отделяется от остальных наименьший элемент. После этого исходный список оказывается изменённым. Изменённый список принимается за исходный и процесс продолжается до тех пор, пока все элементы не будут выбраны. Очевидно, что выбранные элементы образуют упорядоченный список.

Например, требуется найти минимальный элемент списка:

{5, 11, 6, 4, 9, 2, 15, 7}

Процесс выбора показан на рис.1, где в каждой строчке выписаны сравниваемые пары. Выбираемые элементы с меньшим весом обведены кружком. Нетрудно видеть, что число сравнений соответствует на рисунке числу строк, а число перемещений – количеству изменений выбранного элемента.

{5, 11, 6, 4, 9, 2, 15, 7}

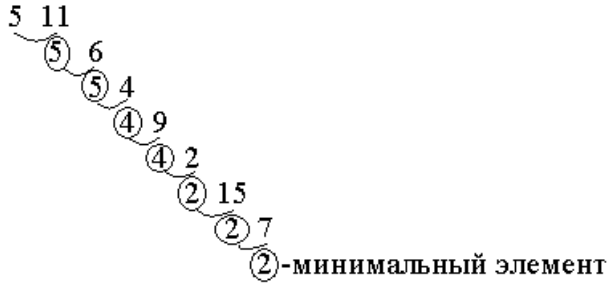


Рис.1. Сортировка выбором

Выбранный в исходном списке минимальный элемент размещается на предназначенном ему месте несколькими способами:

- Минимальный элемент после i-го просмотра перемещается на i-ое место нового списка (i = 1, 2, …. , n), а в исходном списке на место выбранного элемента записывается какое-то очень большое число, превосходящее по величине любой элемент списка, при этом длина заданного списка остаётся постоянной. Изменённый таким образом список можно принимать за исходный.
- Минимальный элемент записывается на i-ое место исходного списка (i = 1, 2, …. , n), а элемент с i-го места - на место выбранного. При этом очевидно, что уже упорядоченные элементы (а они будут расположены, начиная с первого места) исключаются из дальнейшей сортировки, поэтому длина каждого последующего списка (списка, участвующего в каждом последующем просмотре) должна быть на 1 элемент меньше предыдущего.
- Выбранный минимальный элемент, как и в предыдущем случае, перемещается на i-ое место заданного списка, а чтобы это i-ое место освободилось для записи очередного минимального элемента, левая от выбранного элемента часть списка перемещается вправо на одну позицию так, чтобы заполнилось место, занимаемое до этого выбранным элементом.

Сложность метода сортировки выбором порядка $O(n^2)$.

3 Сортировка вставкой

В этом методе из неупорядоченной последовательности элементов выбирается поочередно каждый элемент, сравнивается с предыдущим, уже упорядоченным, и помещается на соответствующее место.

Сортировку вставкой рассмотрим на примере заданной неупорядоченной последовательности элементов:

{40, 11, 83, 57, 32, 21, 75, 64}

Процедура сортировки отражена на рис.2, где кружком на каждом этапе обведён анализируемый элемент, стрелкой сверху отмечено место перемещения анализируемого элемента, в рамку заключены упорядоченные части последовательности.

Этапы:

1-й 40, 11, 83, 57, 32, 21, 75, 64
11, 40, 83, 57, 32, 21, 75, 64

2-й 11, 40, 83, 57, 32, 21, 75, 64
11, 40, 83, 57, 32, 21, 75, 64

3-й 11, 40, 83, 57, 32, 21, 75, 64
11, 40, 57, 83, 32, 21, 75, 64

4-й 11, 40, 57, 83, 32, 21, 75, 64
11, 32, 40, 57, 83, 21, 75, 64

5-й 11, 32, 40, 57, 83, 21, 75, 64
11, 21, 32, 40, 57, 83, 75, 64

6-й 11, 21, 32, 40, 57, 83, 75, 64
11, 21, 32, 40, 57, 75, 83, 64

7-й 11, 21, 32, 40, 57, 75, 83, 64
11, 21, 32, 40, 57, 64, 75, 83

Рис. 2. Сортировка вставкой

На первом этапе сравниваются два начальных элемента. Поскольку второй элемент меньше первого, он перемещается на место первого элемента, который сдвигается вправо на одну позицию. Остальная часть последовательности остаётся без изменения. На втором этапе из неупорядоченной последовательности выбирается элемент и сравнивается с двумя упорядоченными ранее элементами. Так как он больше предыдущих, то остаётся на месте. Затем анализируются четвёртый, пятый и последующие элементы – до тех пор, пока весь список не будет упорядоченным, что имеет место на последнем (седьмом) этапе.

Разновидностью сортировки вставкой является метод **фон Неймана**. Пусть заданы два упорядоченных по возрастанию элементов одномерных массива: a размерности n и b размерности m. Требуется получить третий массив с размерности n+m, который содержал бы все элементы исходных массивов, упорядоченных по возрастанию.

Алгоритм решения этой задачи ,известный как «сортировка фон Неймана» или сортировка слиянием, состоит в следующем: сначала анализируются первые элементы обоих массивов. Меньший элемент переписывается в новый массив. Оставшийся элемент последовательно сравнивается с элементами из другого массива. В новый массив после каждого сравнения попадает меньший элемент. Процесс продолжается до исчерпания элементов одного из массивов. Затем остаток другого массива дописывается в новый массив. Полученный новый массив упорядочен таким же образом, как исходные.

Сложность метода сортировки вставкой порядка $O(n^2)$.

Сортировка обменом

Сортировка обменом – метод, в котором элементы списка последовательно сравниваются между собой и меняются местами в том случае, если предшествующий элемент больше последующего.

Требуется, например, провести сортировку списка методом стандартного обмена или методом ’пузырька’ :

{40, 11, 83, 57, 32, 21, 75, 64}

Обозначим квадратными скобками со стрелками обмениваемые элементы, а - сравниваемые элементы. Первый этап сортировки показан на рис.3, а второй этап – на рис.4.

Нетрудно видеть, что после каждого просмотра списка все элементы, начиная с последнего, занимают свои окончательные позиции, поэтому их не следует проверять при следующих просмотрах. Каждый последующий просмотр исключает очередную позицию с найденным максимальным элементом, тем самым укорачивая список. После первого просмотра в последней позиции оказался больший элемент, равный 83 (исключаем его из дальнейшего рассмотрения).

Второй просмотр выявляет максимальный элемент, равный 75 (рис.4).

Процесс сортировки продолжается до тех пор, пока не будут сформированы все элементы конечного списка, либо не выполнится условие Айверсона.

Исходный список	40	11	83	57	32	21	75	64
Первый просмотр	11	40 40	83 57	83 32	83 21	83 75	83 64	83
Полученный список	11	40	57	32	21	75	64	83

Рис. 3. Сортировка обменом (первый просмотр)

Исходный список	11	40	57	32	21	75	64
Второй просмотр	11	40 40	57 32	57 21	57 57	75 64	75
Полученный список	11	40	32	21	57	64	75

Рис. 4. Сортировка обменом (второй просмотр)

<u>Условие Айверсона:</u> если в ходе сортировки при сравнении элементов не было сделано ни одной перестановки, то множество считается упорядоченным (условие Айверсона выполняется только при шаге d=1).

Модификацией сортировки стандартным обменом является **шейкерная** или челночная сортировка . Здесь, как и в методе пузырька проводится по парное сравнение элементов . При этом первый проход осуществляется слева направо, второй – справа налево и т.д. Иными словами меняется направление просмотра элементов списка.

Сложность метода стандартного обмена $O(n^2)$.

4Сортировка Шелла

В методе Шелла сравниваются не соседние элементы, а элементы, расположенные на расстоянии d (где d – шаг между сравниваемыми элементами) d = [n/2]. После каждого просмотра шаг d уменьшается вдвое. На последнем просмотре он сокращается до d = 1.

Например, пусть дан список, в котором число элементов чётно:

{40, 11, 83, 57, 32, 21, 75, 64}

Список длины n разбивается на n/2 частей, т.е. d = [n/2] = 4.

Исходный массив	40	11	83	57	32	21	75	64
d = 4	32	11	75	57	40	21	83	64
Полученный массив	32	11	75	57	40	21	83	64

Рис. 5. Метод Шелла (d=4)

Исходный массив	32	11	75	57	40	21	83	64
d = 2	32	11	40	21	75	57	83	64
Полученный массив	32	11	40	21	75	57	83	64

Рис. 6. Метод Шелла (d=2)

При первом просмотре сравниваются элементы, отстоящие друг от друга на d = 4 (рис.5), т.е. k_1 и k_5, k_2 и k_6 и т.д. Если $k_i > k_{i+d}$, то происходит обмен между позициями i и (i+d). Перед вторым просмотром выбирается шаг d = [d / 2] = 2 (рис.6). Затем выбираем шаг d = [d / 2] = 1 (рис.7), т.е. имеем аналогию с методом стандартного обмена.

Сложность метода Шелла $O(0{,}3n(\log_2 n)^2)$.

Исходный список	32	11	40	21	75	57	83	64
d =1	11	32	21	40	57	75	64	83
Полученный список	11	32	21	40	57	75	64	83

Рис. 7. Метод Шелла (d=1)

5Быстрая сортировка (сортировка Хоара)

В методе быстрой сортировки фиксируется какой-либо ключ (базовый), относительно которого все элементы с большим весом перебрасываются вправо, а с меньшим – влево. При этом весь список элементов делится относительно базового ключа на две части. Для каждой части процесс повторяется.

Поясним метод на примере.

На рис.8 представлен первый этап быстрой сортировки. В первой строке указана исходная последовательность.

Примем первый элемент последовательности за базовый ключ, выделим его квадратом и обозначим $k_0 = 40$. Установим два указателя : i и j, из которых i начинает отсчёт слева (i=1), а j – справа (j=n).

Сравниваем базовый ключ k_0 и текущий ключ k_j. Если $k_0 <= k_j$, то устанавливаем j=j-1 и проводим следующее сравнение k_0 и k_j. Продолжаем уменьшать j до тех пор, пока не достигнем условия $k_0 > k_J$. После этого меняем местами ключи k_0 и k_j (шаг 3 на рис.8).

Номер шага	i →					← j			Примечание
	40	11	83	57	32	21	75	64	Исходный список
1	40							64	k0<kj
2	40						75		k0<kj
3	40 21						21 40		Обмен; k0>kj
4		11					40		ki<k0
5			83 40				40 83		Обмен; ki>k0
6			40 32			32 40			Обмен; k0>kj
7				57 40	40 57				Обмен; ki>k0
	21	11	32	40	57	83	75	64	Полученный список

Рис. 8. Метод Хоара

Теперь начинаем изменять индекс i = i + 1 и сравнивать элементы k_i и k_0. Продолжаем увеличение i до тех пор, пока не получим условие $k_i > k_0$, после чего следует обмен k_i и k_0 (см.шаг5). Снова возвращаемся к индексу j, уменьшаем его. Чередуя уменьшение j и увеличение i, продолжаем этот процесс с обоих концов к середине до тех пор, пока не получим i = j (см.шаг 7).

В отличие от предыдущих рассмотренных сортировок уже на первом этапе имеют место два факта: во-первых, базовый ключ $k_0 = 40$ занял своё постоянное место в сортируемой последовательности; во-вторых, все элементы слева от k_0 будут меньше него, а справа-больше него. Таким образом, по окончании первого этапа имеем:

21, 11, 32 **40** ***57, 83, 75, 64***

левая часть правая часть

Указанная процедура сортировки применяется независимо к левой и правой частям.
Сложность метода Хоара $O(n \log_2 n)$.

6Сортировка в нелинейных структурах

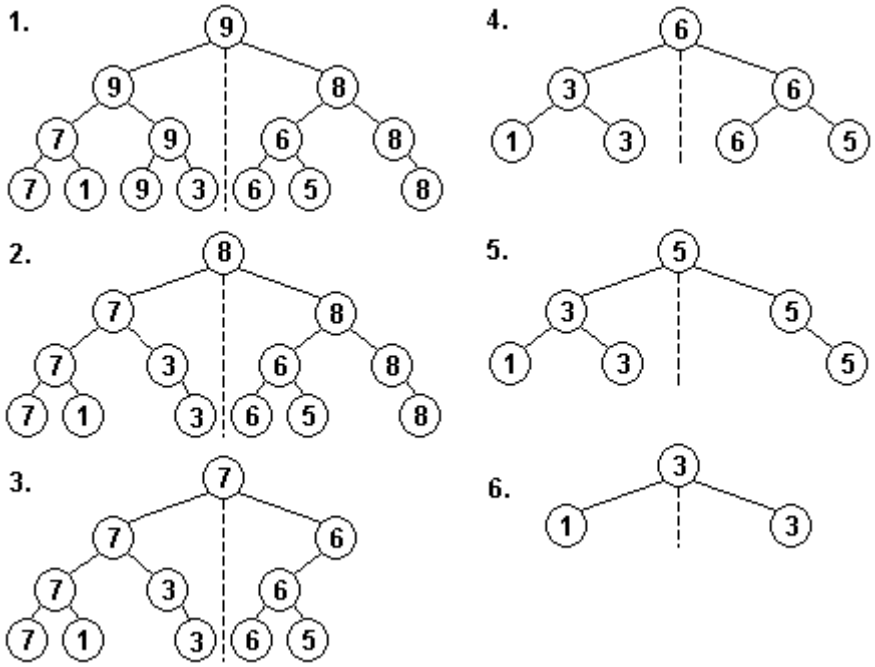
Сортировка в нелинейных структурах осуществляется только на бинарных деревьях (деревьях, из каждой вершины которого выходит по два ребра).

6.1 Турнирная сортировка

Свое название эта сортировка получила, потому что она используется при проведении соревнований, турниров и олимпиад. Элементы исходного множества представляются листьями дерева. Их по парное сравнение позволяет определить максимальный элемент.

Пример: Дано исходное множество { 7, 1, 9, 3, 6, 5, 8 }

Производится по парное сравнение элементов снизу- вверх. Найденный максимальный элемент помещается в результирующее множество.



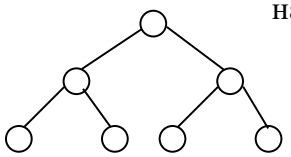
В результате будет получено упорядоченное множество { 9,8,7,6,5,3}

6.2 Пирамидальная сортировка

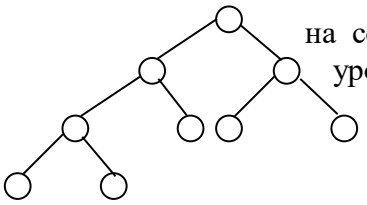
Данный тип сортировки заключается в построение пирамидального дерева.

Пирамидальное дерево – это бинарное дерево обладающее тремя свойствами:

- В вершине каждой триады располагается элемент с большим весом.
- Листья бинарного дерева располагаются либо в одном уровне либо в двух соседних



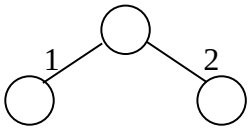
на одном уровне



на соседних уровнях

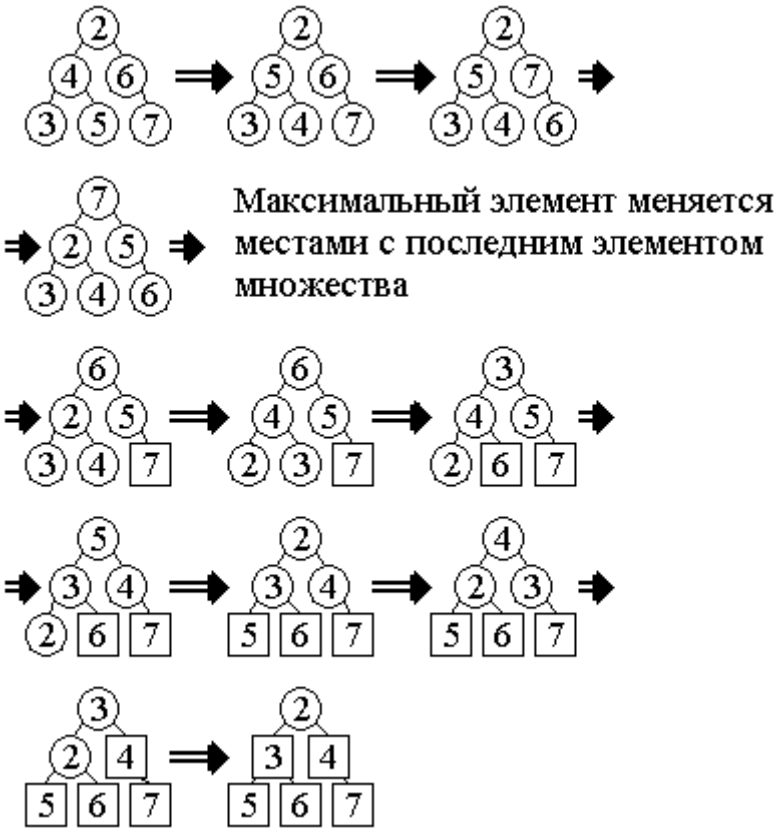
- Листья нижнего уровня располагаются левее листьев более высокого уровня.

В ходе преобразования элементы триад сравниваются дважды , при этом элемент с большим весом перейдет вверх, а с меньшим - вниз.



1 - первое сравнение
2 - второе сравнение

Пример: Дано исходное множество { 2, 4, 6, 3, 5, 7 }

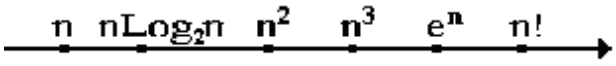


В результате будет получено упорядоченное множество { 2,3,4,5,6,7 }

7Функция сложности алгоритма

Для оценки эффективности алгоритмов используется функция сложности алгоритма, которая обозначается заглавной буквой “O”, в круглых скобках записывается аргумент. Например, функция сложности $O(n^2)$ читается как функция сложности порядка n^2. Функция сложности алгоритма – это функция, которая определяет количество сравнений, перестановок а так временные и ресурсные затраты на реализацию алгоритма.

Функция сложности принимает следующий ряд значений:



Функция сложности

Чем правее на оси расположена функция сложности, тем сложнее алгоритм.

Лабораторное задание

Для каждого из перечисленных методов сортировки провести анализ временных затрат для списков различной размерности.

8Методика выполнения лабораторной работы

Для проведения лабораторной работы необходимо выполнить следующие действия.

1. Вызвать систему **Sort_new**, включающую в себя сортировку неупорядоченных списков в линейных и нелинейных структурах.

Путь к файлу: D:\ИПОВС\АиСД\SORT\Sort_new.exe

Система работает в диалоговом режиме с использованием “меню”. Вся необходимая информация во время работы системы отображается на экране дисплея и не требует специальных пояснений.

Основные функции системы

K - конец работы;
? - выдача краткого сообщения о командах;
U – задание условий для генерации неупорядоченного массива;
G - генерации неупорядоченного массива;
P – вывод массива;
W – вывод времени сортировки, количества элементов массива.

Сортировка методом:

1 – простого обмена;
2 – бинарной вставки;
3 – простой вставки;
4 – челночной;
5 – простого выбора;
6 – слиянием;
7 – Шелла;
8 – Хоара;
9 – пирамиды;

Esc – аварийное прерывание выполнения функции.

Краткое описание функции

U– задание условий для генерации неупорядоченного массива (задаются число элементов и границы элементов неупорядоченного массива). Заданные условия сохраняются до следующего вызова функции “U”;
G – генерация неупорядоченного массива, необходима перед каждой сортировкой;
P – вывод массива в текущем состоянии : неупорядоченный (до сортировки) или упорядоченный (после неё);
W - вывод времени сортировки ;
I – 9 – методы сортировки;

Для указанных в лабораторной работе методов сортировки провести следующие исследования:

а) выполнить сортировку массива из N чисел (варианты заданий даны в приложении ; номер варианта соответствует порядковому номеру фамилии студента в списке группы). Результаты занести в таблицу 1.

Таблица 1

Метод			
Количество элементов сортируемого массива N	N_1	N_2	N_3
Время сортировки t, с	t_1	t_2	t_3

2. Провести исследование методов сортировки упорядоченных списков с использованием программы **SORTALL.**
Путь к файлу: D:\ИПОВС\АиСД\SORT\Sortall.exe
Результаты занести в таблицу 1.

3. Провести исследование методов сортировки с использованием программ **WinSort** и **Sort.**
4. Оценить сложность рассмотренных методов сортировки;
   а) провести анализ отклонения полученной в результате эксперимента сложности алгоритма от теоретической;
   б) построить графические зависимости времени сортировки от количества элементов сортируемого массива.

Требования к отчёту
Отчёт должен содержать:
1) конспект лабораторной работы;
2) примеры сортировки ;
3) результаты выполнения работы;
4) выводы по работе;

Контрольные вопросы

1. Что понимается под сортировкой?
2. Каковы особенности сортировки: вставкой, выбором, обменом , Шелла, Хоара, турнирной, пирамидой?
3. Что включает в себя понятие сложности алгоритма?
4. В чём состоит методика анализа сложности алгоритмов сортировки?

Литература
1. Колдаев В.Д., Поддубная Л. М., Полосухин Б.М. Методы сортировки . М.:МИЭТ, 1985.
2. Колдаев В.Д., Поддубная Л. М., Полосухин Б.М. Лабораторный практикум по курсу « Теория алгоритов и вычислительные методы » М.:МИЭТ, 1988
3. Кнут Д. Искусство программирования для ЭВМ. Т. 3. Сортировка и поиск. М. : Мир, 2000.
4. Т. Кормен, Ч. Лейзерсон, Р. Ривест «Алгоритмы: построение и анализ». М.: МЦНМО, 2000.
5. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
6. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на языке Си. Учеб. пособие. М : Финансы и статистика, 2004.

Приложение

Варианты заданий

Вариант	n1	n2	n3	Составить программу
1,15	1000	4000	6000	Простая вставка
2,16	800	3000	7000	Сортировка слиянием
3,17	2000	5000	6800	Метод Шелла
4,18	1500	3500	6000	Простой выбор
5,19	1000	2800	8500	Метод Хоара
6,20	500	2500	6500	Бинарная вставка
7,21	900	3000	8500	Шейкерная сортировка
8,22	1200	2000	7500	Простая вставка
9,23	2500	3500	6500	Шейкерная сортировка
10,24	1600	2800	8800	Метод Шелла
11,25	2200	3400	7200	Простой выбор
12,26	1900	2700	8000	Метод Хоара
13,27	1300	3500	6000	Бинарная вставка
14,28	1700	2800	7500	Сортировка слиянием

Предусмотреть в программе учет времени сортировки для указанных значений n1, n2, n3. Для сортировок Шелла, Хоара, пирамидальной значения n1, n2, n3 удвоить. Составить программу, реализующую один из методов сортировки. Произвести расчет функции сложности разработанного алгоритма по полученным временным затратам.

9 *Пояснения к выполнению работы.*

Определить время выполнения какого-либо фрагмента программы можно с помощью библиотечной функции **gettime()**, прототип которой находится в заголовочном файле **dos.h**. Эта функция должна получать аргумент – адрес структуры типа **time**, в поля которой и заносится информация о текущем времени. В обобщенном виде программа, в которой выполняется контроль времени может выглядеть следующим образом:

```
#include <dos.h>
void main()
{
    time t1,t2;
    gettime(&t1);
// Фрагмент, время работы которого измеряется.
    gettime(&t2);
// ...
}
```

Для получения интервала времени следует вычесть поля структуры **t1** из полей структуры **t2**. Представить этот интервал в секундах можно следующим образом:

```
double delta_t=(t2.ti_hour*360000.+t2.ti_min*6000.+
    t2.ti_sec*100.+t2.ti_hund-
    (t1.ti_hour*360000.+t1.ti_min*6000.+
    t1.ti_sec*100.+t1.ti_hund))/100.;
```

Современные процессоры выполняют сортировку небольших массивов за время, значительно меньшее, чем тысячная доля секунды (**t1.ti_hund**), поэтому для получения точного результата нужно выполнить сортировку достаточно большое число раз, например, миллион. При этом необходимо каждый раз сортировать один и тот же неупорядоченный массив. Лучше сортировать второй массив, каждый раз перед сортировкой копируя в него данные из исходного неупорядоченного массива. Копирование занимает некоторое время, которое необходимо учесть.

После завершения всего процесса следует проверить правильность сортировки, выведя содержимое второго массива на экран.

Итоговая программа в обобщенном виде может выглядеть следующим образом:

```
#include <dos.h>
#include <iostream.h>
const int N = 5; // Размер массива.
const unsigned long NN = 1000000; // Число сортировок.
void main()
{
    char arr1[N]={ /* Элементы неупорядоченного массива */ };
    char arr[N];
    time t1,t2;
    double t_copy,t_sort;

    gettime(&t1);
// Копирование из массива arr1 в массив arr.
    gettime(&t2);
    t_copy = (t2.ti_hour*360000.+t2.ti_min*6000.+
        t2.ti_sec*100.+t2.ti_hund-(t1.ti_hour*360000.+
        t1.ti_min*6000.+t1.ti_sec*100.+t1.ti_hund))/100.;

    gettime(&t1);
// Цикл сортировки из большого числа проходов.
    gettime(&t2);

    t_sort = (t2.ti_hour*360000.+t2.ti_min*6000.+
        t2.ti_sec*100.+t2.ti_hund-(t1.ti_hour*360000.+
        t1.ti_min*6000.+t1.ti_sec*100.+t1.ti_hund))/100.;
    t_sort-=t_copy;
// Контроль содержимого массива arr после сортировки.
// Вывод на экран времени сортировки t_sort.
}
```

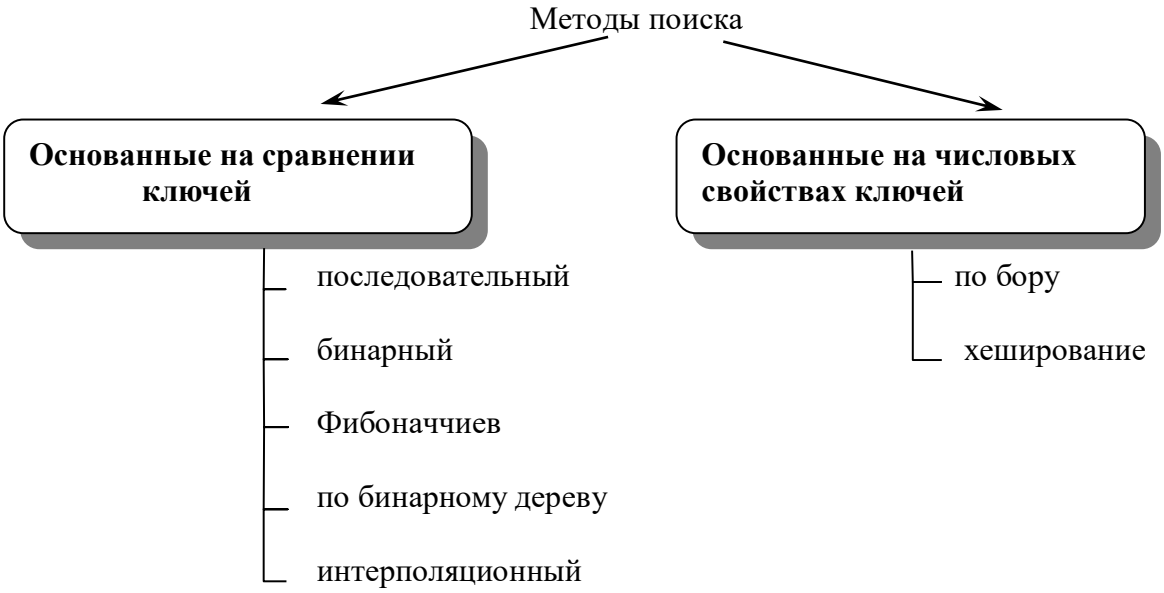
1 Лабораторная работа -Методы поиска

<u>Цель работы</u>: ознакомление с алгоритмами поиска в линейных и нелинейных структурах и оценкой эффективности алгоритмов.
<u>Продолжительность работы</u>: - 2 часа.

10 Теоретические сведения

Предметы (объекты), составляющие множество, называются его элементами. Элемент множества будет называться ключом, и обозначаться латинской буквой “k” с индексом, указывающим номер элемента.

Алгоритмы поиска можно разбить на следующие группы:



Задача поиска:

Пусть дано множество ключей **{k_1, k_2, k_3...k_n}**.
Необходимо отыскать во множестве ключ **k_i**. Поиск может быть завершён в двух случаях: 1) Ключ во множестве отсутствует;
2) Ключ найден во множестве.

11 Последовательный поиск.

В последовательном поиске исходное множество не упорядоченно, т.е. имеется произвольный набор ключей **{k_1, k_2, k_3...k_n}**. Метод заключается в том, что отыскиваемый ключ **k_i** последовательно сравнивается со всеми элементами множества. При этом поиск заканчивается досрочно, если ключ найден.

12 Бинарный поиск.

В бинарном поиске исходящее множество должно быть упорядоченно по возрастанию. Иными словами каждый последующий ключ больше предыдущего

$$\{k_1 \le k_2 \le k_3 \le k_4 ... k_{n-1} \le k_n\}.$$

Отыскиваемый ключ сравнивается с центральным элементом множества, если он меньше центрального, то поиск продолжается в левом подмножестве, в противном случае в правом.
Центральный элемент находится по формуле N эл-та =[n/2]+1,
где квадратные скобки обозначают, что от деления берётся только целая часть (всегда округляется в меньшую сторону). В методе бинарного поиска анализируются только центральные элементы.
Пример. Дано множество
{7,8,12,16,18,20,30,38,49,50,54,60,61,69,75,79,80,81,95,101,123,198}
Найти во множестве ключ K=61.

Шаг 1
N эл-та =[n/2]+1=[22/2]+1=12
$K \sim k_{12}$
61>60 Дальнейший поиск в правом подмножестве {61,69,75,79,80,81,95,101,123,198}.
Значок ”~” обозначает сравнение элементов (чисел, значений).

Шаг 2
N эл-та =[n/2]+1=[12/2]+1=7
$K \sim k_{19}$
61<95 Дальнейший поиск в левом подмножестве {61,69,75,79,80,81} (относительно предыдущего подмножества).

Шаг 3
N эл-та =[n/2]+1=[6/2]+1=4
$K \sim k_{16}$
61<79 Дальнейший поиск в левом подмножестве {61,69,75,79}.

Шаг 4
N эл-та =[n/2]+1=[4/2]+1=3
$K \sim k_{15}$
61<75 Дальнейший поиск в левом подмножестве {61,69} .

Шаг 5
N эл-та =[n/2]+1=[2/2]+1=2
$K \sim k_{14}$
61<69 Дальнейший поиск в левом подмножестве.

Шаг 6
$K \sim k_{13}$
61=61.
Вывод: искомый ключ найден под номером 13.

13 Фибоначчиев поиск.

В этом поиске анализируются элементы, находящиеся в позициях, равных числам Фибоначчи. Числа Фибоначчи получаются по следующему правилу:
каждое последующее число равно сумме двух предыдущих чисел, например:

{1,2,3,5,8,13,21,34,55,…}.

Поиск продолжается до тех пор, пока не будет найден интервал между двумя ключами, где может располагаться отыскиваемый ключ.
Пример**.** Дано исходное множество ключей

{3,5,8,9,11,14,15,19,21,22,28,33,35,37,42,45,48,52}

Пусть отыскиваемый ключ равен 42. (K=42).
Последовательное сравнение отыскиваемого ключа будет проводиться в позициях, равных числам Фибоначчи: {1,2,3,5,8,13,21,…}

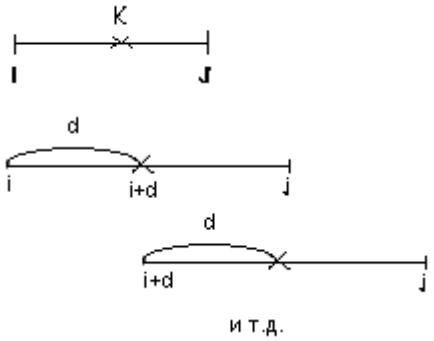
Шаг 1. $K \sim K_1$ 42>3 => отыскиваемый ключ сравнивается с ключом, стоящим в позиции равной числу Фибоначчи.
Шаг 2. $K \sim K_2$ 42>5 => сравнение продолжается с ключом, стоящим в позиции равной следующему числу Фибоначчи.
Шаг 3. $K \sim K_3$ 42>8 => сравнение продолжается
Шаг 4. $K \sim K_5$ 42>11 => сравнение продолжается
Шаг 5. $K \sim K_8$ 42>19 => сравнение продолжается
Шаг 6. $K \sim K_{13}$ 42>35 => сравнение продолжается
Шаг 7. $K \sim K_{18}$ 42<52 => найден интервал, в котором находится отыскиваемый ключ: от 13 до 18 позиции, т.е. {35,37,42,45,48,52}
В найденном интервале поиск вновь ведётся в позициях равных числам Фибоначчи.

14 Интерполяционный поиск.

Исходное множество должно быть упорядочено по возрастанию весов.
Первоначальное сравнение осуществляется на расстоянии шага d , который определяется по формуле:

$$\mathbf{d}=\left[\frac{(j-i)(K-K_i)}{K_j-K_i}\right]$$

Где:
i – номер первого рассматриваемого элемента
j – номер последнего рассматриваемого элемента
K – отыскиваемый ключ
K_i, K_j – значения ключей в i и j позициях
[ ] – целая часть от числа.

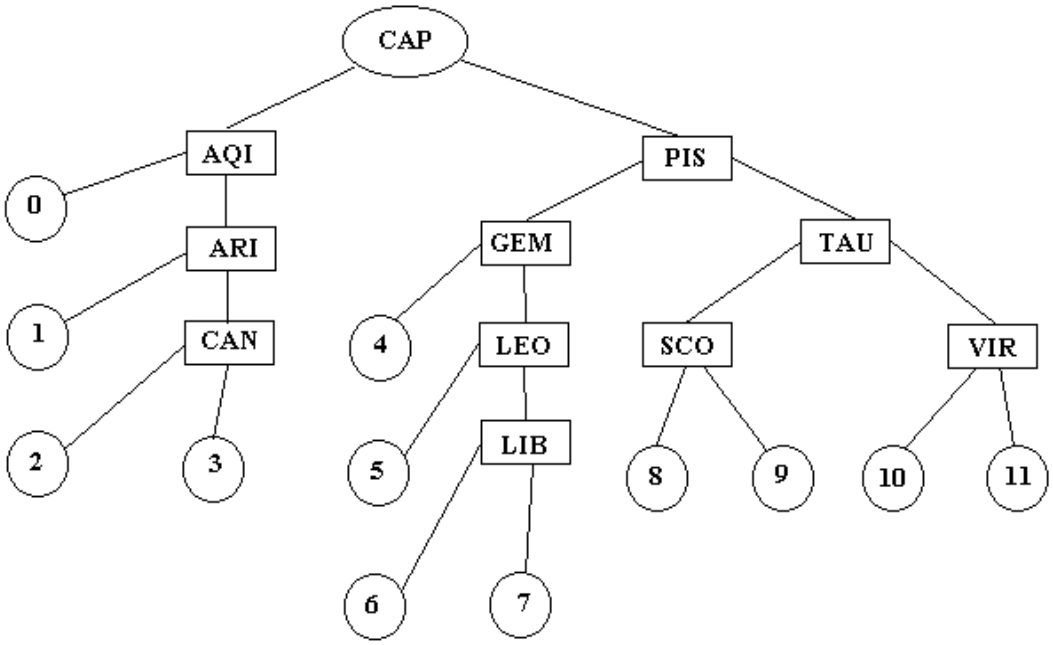


Идея метода заключается в следующем:
шаг d меняется после каждого этапа, по формуле приведённой выше.
Алгоритм заканчивает работу при d=0, при этом анализируются соседние элементы, после чего делается окончательно решение.
Этот метод прекрасно работает, если исходное множество представляет собой арифметическую прогрессию или множество, приближенное к ней.
Пример **.** Дано множество ключей:
{2,9,10,12,20,24,28,30,37,40,45,50,51,60,65,70,74,76}
Пусть искомый ключ равен 70. (K=70).
Шаг 1. Определим шаг d для исходного множества ключей:
d=[(18-1)(70-2)/(76-2)]=15
Сравниваем ключ, стоящий под шестнадцатым порядковым номером в данном множестве с искомым ключом:
$K_{16} \sim K$ 70=70 ключ найден.

15 *Поиск по бинарному дереву.*

Использование структуры бинарного дерева позволяет быстро вставлять и удалять записи и производить эффективный поиск по таблице. Такая гибкость достигается добавлением в каждую запись двух полей для хранения ссылок.

Пусть дано бинарное дерево:



Требуется по бинарному дереву отыскать ключ SAG.

При просмотре от корня дерева видно, что по первой букве латинского алфавита, название SAG больше чем CAP. Следовательно, дальнейший поиск будем осуществлять в правой ветви. Это слово больше чем PIS - снова идем вправо; оно меньше чем TAU - идем влево; оно меньше чем SCO и попадаем в узел 8. Таким образом, название SAG должно находиться в узле 8.

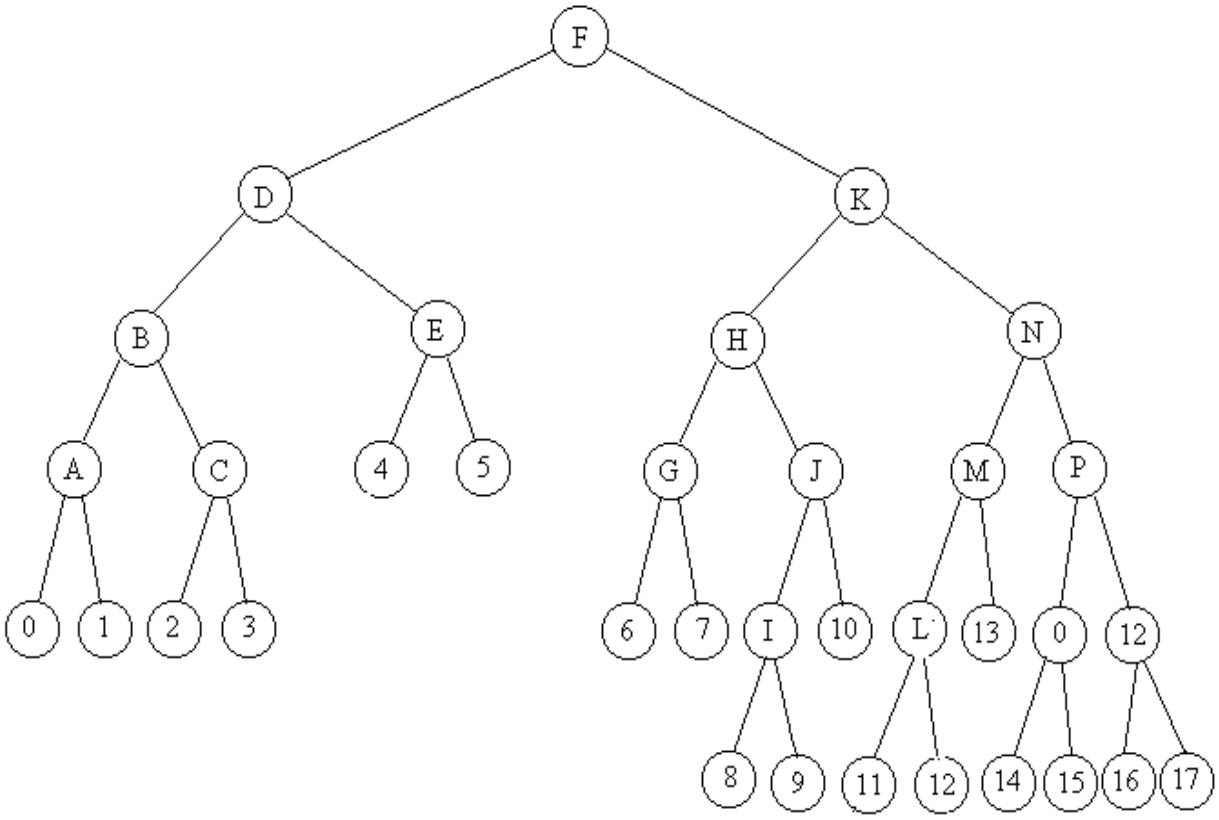
При этом узлы дерева имеют следующую структуру:

Ключ	Информационная часть	Указатель на левое поддерево	Указатель на правое поддерево
	/может отсутствовать/		
KEY		LLINK	RLINK

16 Поиск по сбалансированному дереву.

Бинарное дерево называется сбалансированным, если высота левого поддерева каждого узла отличается от высоты правого не более чем
на ±1.

Например:



Сбалансированные бинарные деревья занимают промежуточное положение между оптимальными бинарными деревьями (все внешние узлы, которых расположены на двух смежных уровнях) и произвольными бинарными деревьями.

Бинарное дерево называется сбалансированным, если высота левого поддерева каждого узла отличается от высоты правого не более чем на ±1.

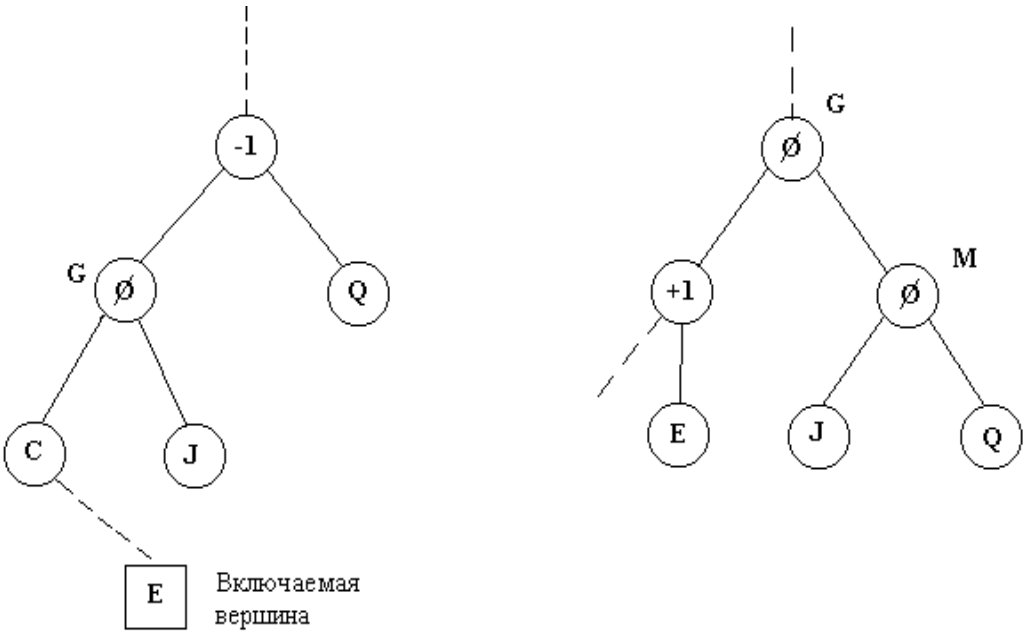
Рассмотрим следующую структуру узлов сбалансированного бинарного дерева:

Ключ	Указатель на левое поддерево	Указатель на правое поддерево	Показатель сбалансированности узла
KEY	LLINK	RLINK	B

где

B – показатель сбалансированности узла, то есть разность высот правого и левого поддерева (B = +1; 0; -1).

При восстановлении баланса дерева по высоте учитывается показатель B.



Символы +1, ∅ ,-1 указывают, что левое поддерево выше правого, поддеревья равны по высоте, правое поддерево выше левого.

17 Поиск по бору

Особую группу методов поиска образует представление ключей в виде последовательности цифр и букв. Рассмотрим, например, имеющиеся во многих словарях буквенные высечки. Тогда по первой букве данного слова можно отыскать страницы, содержащие все слова, начинающиеся с этой буквы. Развивая идею побуквенных высечек, получим схему поиска, основанную на индексации в структуре бора (термин использует часть слова выборка).

Бор представляет собой m –арное дерево. Каждый узел уровня h представляет множество всех ключей, начинающихся с определенной последовательности из h литер. Узел определяет m – путевое разветвление в зависимости от (h + 1) –ой литеры.

Бор представляет собой таблицу, следующего вида:

Узлы / **Символы**	**1**	**2**	**3**	**...**	**N**
Пробел(_)					

Пробел (_)- обязательный символ таблицы.
В первом узле записывается первая буква или цифра ключа. Во втором узле к ней добавляется ещё один символ и т.д. Если слово начинающееся с определенной буквы (цифры) единственное , то оно сразу записывается в первом узле .
Пример. Дано множество
{A,AA,AB,ABC,ABCD,ABCA,ABCC,BOR,C,CC,CCC,CCCD,CCCB,CCCA}
От исходного множества перейдём к построению бора.
Исходный алфавит = {A,B,C,D}
BOR- единственное слово на букву B и оно побуквенно не разбивается.

Узлы / Символы	1	2	3	4	5	6	7
_		A_	AB_	ABC_	C_	CC_	CCC_
A	2	AA		ABCA			CCCA
B	BOR	3					CCCB
C	5		4	ABCC	6	7	
D				ABCD			CCCD

Узлы бора представляют собой векторы, каждая компонента которых представляет собой либо ключ, либо ссылку (возможно пустую).

Узел 1 – корень, и первую букву следует искать здесь. Если первой сказалась, например, буква B, то из таблицы видно, что ему соответствует слово BOR . Если же первая буква A, то первый узел передает управление к узлу 2, где аналогичным образом отыскивается вторая буква. Узел 2 указывает, что вторыми буквами будут _, A, B и т. д.

18 Поиск хешированием

В основе поиска лежит переход от исходного множества к множеству хеш-функций **h(k).** Хеш-функция имеет следующий вид:

h(k)=k mod (m),

где **k**-ключ, **m**- целое число, **mod**-целочисленный остаток от деления.

Например, пусть дано множество {9,1,4,10,8,5}.

Определим для него хеш-функцию h(k)= k mod(m);

- Пусть m=1, тогда
  h(k)={0, 0, 0, 0, 0, 0};
- Пусть m=20, тогда
  h(k)={9, 1, 4, 10, 8, 5};
- Пусть m равно половине максимального ключа, тогда m=[10/2]=5
  h(k)={4, 1, 4, 0, 3, 0};

Хеш-функция указывает адрес, по которому следует отыскивать ключ. Для разных ключей хеш-функция может принимать одинаковые значения, такая ситуация называется коллизией. Таким образом, поиск хешированием заключается в устранении коллизий.

Пример. Дано множество

{7, 13, 6, 3, 9, 4, 8, 5}

Найти ключ K=27.

Хеш-функция равна h(k)= K mod(m);

m=[13/2]=6 (т.к. 13- максимальный ключ)

h(k)={1,1,0,3,3,4,2,5}

Для устранения коллизий построим таблицу.

h(k)	Цепочки ключей
0	6
1	7,12
2	8
3	3, 9
4	4
5	5

По парным сравнением множества хеш-функцийи множества исходных ключей, заполняем таблицу.

и множества исходных ключей, заполняем таблицу.

Напоминаем, что хеш-функция указывает адрес, по которому следует отыскивать ключ.

Например , если отыскивается ключ K=27, тогда

h(k)=27 mod 6=3-это значит, что ключ K=27 может быть только в 3-й строке. Так как его там нет, то данный ключ
отсутствует в исходном множестве .

19 Алгоритмы поиска словесной информации

В настоящее время наличие сверхпроизводительных микропроцессоров и дешевизна электронных компонентов позволяют делать значительные успехи в алгоритмическом моделировании. Рассмотрим несколько алгоритмов обработки слов.

19.1 Алгоритм Кнута - Морриса - Пратта

Алгоритм Кнута-Морриса-Пратта (КМП) получает на вход слово

X=x[1]x[2]... x[n]

и просматривает его слева направо буква за буквой, заполняя при этом массив натуральных чисел l[1]... l[n], где

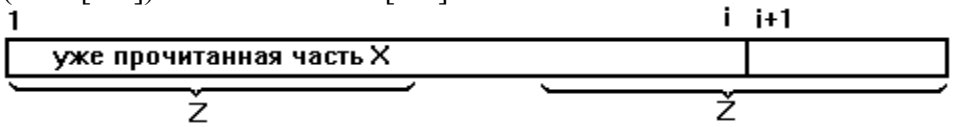
l[i]=длина слова l(x[1]...x[i])

Таким образом: l[i] есть длина наибольшего начала слова x[1]...x[i], одновременно являющегося его концом.

Используя алгоритм КМП определить , является ли слово A подсловом слова B?

Решение. Применим алгоритм КМП к слову A#B, где # - специальная буква, не встречающаяся ни в A, ни в B. Слово A является подсловом слова B тогда и только тогда, когда среди чисел в массиве l будет число, равное длине слова A.

Предположим, что первые i значений l[1]...l[i] уже найдены. Читается очередная буква слова (т.е. x[i+1]) и вычисляется l[i+1].



Другими словами, необходимо определить начала Z слова

x[1]...x[i+1,

одновременно являющиеся его концами -из них следует выбрать самое длинное.

Получаем такой рецепт отыскания слова Z. Рассмотрим все начала слова x[1]...x[i], являющиеся одновременно его концами. Из них выберем подходящие - те, за которыми идет буква x[i+1]. Из подходящих выберем самое длинное. Приписав в его конец x[i+1], получим искомое слово Z. Теперь пора воспользоваться сделанными приготовлениями и вспомнить, что все слова, являющиеся одновременно началами и концами данного слова, можно получить повторными применениями к нему функции l .

Вот что получается:

```
i:=1; 1[1]:=0;
 {таблица l[1]..l[i] заполнена правильно}
 while i <> n do begin
    len:= l[i]
    {len - длина начала слова x[1]..x[i], которое является
    его концом; все более длинные начала оказались
    неподходящими}
    while (x[len+1]<>x[i+1]) and (len>0) do begin
          {начало не подходит, применяем к нему функцию l}
       len:=l[len];
       end;
          {нашли подходящее или убедились в отсутствии}
       if x[len+1]=x[i+1] do begin
```

```
            {x[1]..x[len] - самое длинное подходящее начало}
        l[i+1]:=len+1;
        end else begin
            {подходящих нет}
        l[i+1]:= 0;
    end;
    i:=i+1;
end;
```

Запишем алгоритм проверяющий, является ли слово X=x[1]...x[n] подсловом слова Y=y[1]...y[m]

Решение. Вычисляем таблицу l[1]...l[n] как раньше.

```
j:=0; len:=0;
    {len - длина максимального качала слова X, одновременно
            являющегося концом слова y[1]..j[j]}
while (len<>n) and (j<>m) do begin
        while (x[len+1]<>y[j+1]) and (len>0) do begin
                {начало не подходит, применяем к нему функцию l}
            len: = l[len];
            end;
                {нашли подходящее или убедились в отсутствии}
            if x[len+1]=y[j+1] do begin
                {x[1]..x[len] - самое длинное подходящее начало}
            len:=len+1;
            end else begin
                {подходящих нет}
            len:=0;
        end;
        j:=j+1;
end;
{если len=n, слово X встретилось; иначе мы дошли до конца
слова Y, так и не встретив X}
```

19.2 Алгоритм Бойера - Мура

Этот алгоритм делает то, что на первый взгляд кажется невозможным: в типичной ситуации он читает лишь небольшую часть всех букв слова, в котором ищется заданный образец. Пусть, например, отыскивается образец abcd. Посмотрим на четвертую букву слова: если, к примеру, это буква e, то нет никакой необходимости читать первые три буквы. (В самом деле, в образце буквы e нет, поэтому он может начаться не раньше пятой буквы.)

Приведем самый простой вариант этого алгоритма, который не гарантирует быстрой работы во всех случаях. Пусть x[1]...x[n] - образец, который надо искать. Для каждого символа s найдем самое правое его вхождение в слово X, то есть наибольшее k, при котором x[k]=s. Эти сведения будем хранить в массиве pos[s]; если символ s вовсе не встречается, то будет удобно предположить pos[s]=0 ..

Решение.

положить все pos[s] равными 0

```
        for i:=1 to n do begin
            pos[x[i]]:=i;
        end;
```

В процессе поиска будем хранить в переменной last номер буквы в слове, против которой стоит последняя буква образца. Вначале last=n (длина образца), затем last постепенно увеличивается.

```
last:=n;
   {все предыдущие положения образца уже проверены}
  while last<= m do begin {слово не кончилось}
   if x[m]<>y[last] then begin {последние буквы разные}
     last:=last+(n-pos[y[last]]);
    {n - pos[y[last]] - это минимальный сдвиг образца,
          при котором напротив y[last] встанет такая же
          буква в образце. Если такой буквы нет вообще,
          то сдвигаем на всю длину образца}
   end else begin
       если нынешнее положение подходит, т.е. если
       x[i]..x[n]=y[last-n+1]..y[last],
       то сообщить о совпадении;
       last:=last+1;
   end;
end;
```

19.3 Алгоритм Рабина

Этот алгоритм основан на простой идее. Представим себе, что в слове длины m ищется образец длины n. Вырежем окошко размера n и будем двигать его по входному слову. При этом проверяем, не совпадает ли слово в окошке с заданным образцом. Сравнивать по буквам долго. Вместо этого фиксируем некоторую функцию, определенную на словах длины n. Если значения этой функции на слове в окошке и на образце различны, то совпадения нет. Только если значения одинаковы, нужно проверять совпадение по буквам.

В чем выигрыш при таком подходе. Ведь чтобы вычислить значение функции на слове в окошке, все равно нужно прочесть все буквы этого слова. Так уж лучше их сразу сравнить с образцом. Тем не менее выигрыш возможен, так как при сдвиге окошка слово не меняется полностью, а лишь добавляется буква в конце и убирается в начале. Хорошо бы, чтобы по этим данным можно было рассчитать, как меняется функция.

Заменим все буквы в слове и образце их номерами, представляющими собой целые числа. Тогда удобной функцией является сумма цифр. (При сдвиге окошка нужно добавить новое число и вычесть пропавшее.)

Выберем некоторое число p (желательно простое) и некоторый вычет x по модулю p. Каждое слово длины n будем рассматривать как последовательность целых чисел (заменив буквы кодами). Эти числа будем рассматривать как коэффициенты многочлена степени n-1 и вычислим значение этого многочлена по модулю p в точке x. Это и будет одна из функций семейства (для каждой пары p и x получается, таким образом, своя функция). Сдвиг окошка на 1 соответствует вычитанию старшего члена (x^{n-1} следует вычислить заранее), умножению на x и добавлению свободного члена.

Следующее соображение говорит в пользу того, что совпадения не слишком вероятны. Пусть число p фиксировано и к тому же простое, а X и Y - два различных слова длины n. Тогда им соответствуют различные многочлены (предполагаем, что коды всех букв различны - это возможно, если p больше числа букв алфавита). Совпадение значений функции означает, что в точке x эти два различных многочлена совпадают, то есть их разность обращается в 0. Разность есть многочлен степени n-1 и имеет не более n-1 корней. Таким образом, если и много меньше p, то случайному x мало шансов попасть в неудачную точку.

Лабораторное задание

Для каждого из перечисленных методов поиска провести анализ временных затрат для списков различной размерности.

20 Методика выполнения лабораторной работы

Для проведения лабораторной работы необходимо выполнить следующие действия.

1. Выполнить тестовый пример поиска

а) выполнить поиск в массиве из N чисел (варианты заданий даны в приложении ; номер варианта соответствует порядковому номеру фамилии студента в списке группы). Результаты занести в таблицу 1.

Таблица1

Метод			
Количество элементов массива N	N_1	N_2	N_3
Время поиска t, с	t_1	t_2	t_3

б) оценить сложность рассмотренных методов поиска;

в) провести анализ отклонения полученной в результате эксперимента сложности алгоритма от теоретической;

г) построить графические зависимости времени поиска от количества элементов массива.

2. Исследовать методы поиска
3. Провести исследование метода хеширования

Требования к отчёту

Отчёт должен содержать:

1) конспект лабораторной работы;
2) примеры методов поиска ;
3) результаты выполнения работы;
4) выводы по работе;

Контрольные вопросы

1. Что понимается под поиском?
2. Каковы особенности поиска: последовательного, бинарного, интерполяционного, Фибоначчиевого, по бинарному дереву , по бору , хешированием ;
3. В чём состоит методика анализа сложности алгоритмов поиска?

Рекомендуемая литература:

1. Кнут Д. Искусство программирования для ЭВМ. Т. 3. Сортировка и поиск. М. : Мир, 2000.
2. Т. Кормен, Ч. Лейзерсон, Р. Ривест «Алгоритмы: построение и анализ». М.: МЦНМО, 2000.

3. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
4. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на языке Си. Учеб. пособие. М : Финансы и статистика, 2004.

Приложение

Варианты заданий

Провести анализ методов поиска для списков различной размерности и построить зависимости времени от числа элементов исходного множества.

Вариант	n1	n2	n3
1,15	1000	4000	6000
2,16	800	3000	7000
3,17	2000	5000	6800
4,18	1500	3500	6000
5,19	1000	2800	8500
6,20	500	2500	6500
7,21	900	3000	8500
8,22	1200	2000	7500
9,23	2500	3500	6500
10,24	1600	2800	8800
11,25	700	3400	7200
12,26	1900	2700	8000
13,27	1300	3500	6000
14,28	1700	2800	7500

Вариант	Составить программу
1,15	Бинарный поиск
2,16	Интерполяционный поиск
3,17	Поиск хешированием
4,18	Фибоначчиев поиск
5,19	Поиск по бинарному дереву
6,20	Поиск по бору
7,21	Фибоначчиев поиск
8,22	Поиск по бору
9,23	Интерполяционный поиск
10,24	Поиск по бору
11,25	Поиск по бинарному дереву
12,26	Поиск хешированием
13,27	Бинарный поиск
14,28	Фибоначчиев поиск

Использовать алгоритмы Кнута - Морриса – Пратта, Бойера – Мура, Рабина для поиска текстовой информации.

2 Лабораторная работа -Итеративные и рекурсивные алгоритмы

Цель работы: изучить рекурсивные алгоритмы и рекурсивные структуры данных; научиться проводить анализ итеративных и рекурсивных процедур; исследовать эффективность итеративных и рекурсивных процедур при реализации на ПЭВМ.
Продолжительность работы: 2 часа.

21 Теоретические сведения

Эффективным средством программирования для некоторого класса задач является рекурсия. С ее помощью можно решать сложные задачи численного анализа, комбинаторики, алгоритмов трансляции, операций над списковыми структурами и т.д. Программы в этом случае имеют небольшие объемы по сравнению с итерацией и требуют меньше времени на отладку.

Под рекурсией понимают способ задания функции через саму себя, например способ задания факториала в виде

N!=(n-1)!*n

В программировании под рекурсивной процедурой (функцией) понимают способ обращения процедуры (функции) к самой себе.

Под итерацией понимают результат многократно повторяемой какой-либо операции, например представление факториала в виде: .

n!=1*2*3... *n.

Среди широкого класса задач удобно представлять с использованием рекурсивных процедур (функций) те задачи, которые сводятся на подзадачи того же типа, но меньшей размерности.

Общая методика анализа рекурсии содержит три этапа:

1. Параметризация задачи, заключающаяся в выделении различных элементов, от которых зависит решение, в частности размерности решаемой задачи. После каждого рекурсивного вызова размерность должна убывать.
2. Поиск тривиального случая и его решение. Как правило, это ключевой этап в рекурсии, размерность задачи при этом часто равна 0 или I.
3. Декомпозиция общего случая, имеющая целью привести задачу к одной или нескольким задачам того же типе, но меньшей размерности.

Рассмотрим понятие итеративного и рекурсивного алгоритмов на примере вычисления факториала.

22 Итеративный алгоритм.

Наиболее простой и естественной формой представления итеративного алгоритма при реализации на ПЭВМ является описание его с использованием цикла. Программа итеративного алгоритма вычисления факториала представлена на рис.1. Программа состоит из процедуры-функции FACTORIAL и основной программы. В основной программе происходит ввод значения N, вызов процедуры-функции и печать результата.

22.1 Итеративное вычисление факториала

```
PROGRAM FI;
VAR
FAC:   LONGINT;
N:  INTEGER;
{  ФУНКЦИЯ  ВЫЧИСЛЕНИЯ  ФАКТОРИАЛА }
FUNCTION   FACTORIAL (N: INTEGER): LONGINT;
```

```
        VAR    F:  LONGINT;
        I:   INTEGER;
      BEGIN
        IF   (N=O) OR  (N=1) THEN   FACTORIAL:=1 ELSE
        BEGIN
        F:= 1;
        FOR   I: = 2 TO N DO
        F: = F * I;
        FACTORIAL:=F;
        END;
     END;
{   ОСНОВНАЯ   ПРОГРАММА }
BEGIN
WRITELN ('ВВЕДИТЕ   ЗНАЧЕНИЕ N')
READLN (N);
FAC:= FACTORIAL(N);      {  ВЫЗОВ ФУНКЦИИ   FACTORIAL   }
WRITELN('  ФАКТОРИАЛ =' , FAC) ;
READLN;
END.
```

Рекурсивный алгоритм. Рекурсивное представление факториала имеет вид:

n! = (n-1)! * n

Проведем анализ этого выражения согласно методике.

1. Параметризация. В данном случае имеется всего один параметр N- целое число.
2. Поиск тривиального случая. При n=0 или n=1 значение факториала равно 1, что соответствует выходу из рекурсии.
3. Декомпозиция общего случая: n! вычисляется через меньшую размерность этой же задачи (n-1)!

Программа рекурсивного алгоритма вычисления факториала представлена ниже.

22.2 Рекурсивное вычисление факториала

```
PROGRAM FR ;
VAR
fac: longint;
n: integer;
{Функция вычисления факториала}
    FUNCTION factorial (n: integer): longint ;
    Begin
    If (n=0) or (n=1) then factorial: =1
    Else factorial: = factorial (n-1)*n;
    End;

{   ОСНОВНАЯ   ПРОГРАММА }
BEGIN
Writeln (' введите значение n');
Readln (n);
fac: = factorial (n); { ВЫЗОВ ФУНКЦИИ   FACTORIAL   }
Writeln ('факториал = ', fac);
Readln;
END.
```

Процедура-функция имеет следующие особенности:

- при вычислении факториала происходит обращение функции к самой себе (подчеркнуто в выражении), но с меньшим значением аргумента N-1 по сравнению с первым вызовом N.

FACTORIAL N: = FACTORIAL (N-1)*N

- при вычислении факториала не используется цикл, что является существенной особенностью рекурсивного алгоритма.

Рассмотрим последовательность действий при рекурсивном вычислении факториала для n= 3:

I) внешний вызов из основной программы FACTORIAL (3);
2) первый рекурсивный вызов FACTORIAL (2);
в операторе FACTORIAL: = FACTORIAL (N-1) *n,
где не происходят никакие вычисления - только вызов (подчеркнуто);
3) второй рекурсивный вызов FACTORIAL (1);
4) получение значения FACTORIAL (1): =1;
5) возврат из второго рекурсивного вызова и вычисление факториала FACTORIAL (2): = 1*2 =2;
6) возврат из первого рекурсивного вызова и вычисление факториала FACTORIAL (3): =2*3=6;
7) возврат в основную программу FAC: =6.

В программу рекурсивного вычисления факториала можно добавить стандартную функцию текущего времени GETTIME(Var Hour, Minute, Second, Sec100: WORD).
Эта процедура может быть использована при подключении модуля DOS (USES DOS).

23 Рекурсивные структуры данных

Список (list) – набор элементов, расположенных в определенном порядке.

Список очередности (pushup list) – список, в котором последний поступающий элемент добавляется к нижней части списка.

Список с использованием указателей (linked list) – список, в котором каждый элемент содержит указатель на следующий элемент списка.

Однонаправленный и двунаправленный список – это линейный список, в котором все исключения и добавления происходят в любом месте списка.

Однонаправленный список отличается от двунаправленного списка только связью. То есть в однонаправленном списке можно перемещаться только в одном направлении (из начала в конец), а двунаправленном – в любом(рис.1.)

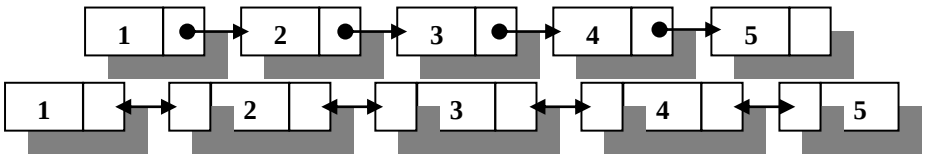


Рис.1 Однонаправленный и двунаправленный список

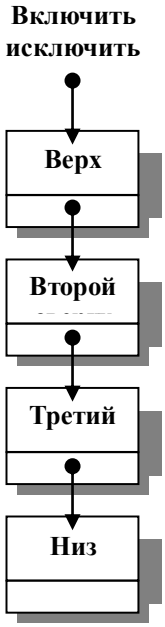
В однонаправленном списке структура добавления и удаления такая же только связь между элементами односторонняя.

Очередь — тип данных, при котором новые данные располагаются следом за существующими в порядке поступления; поступившие первыми данные при этом обрабатываются первыми.

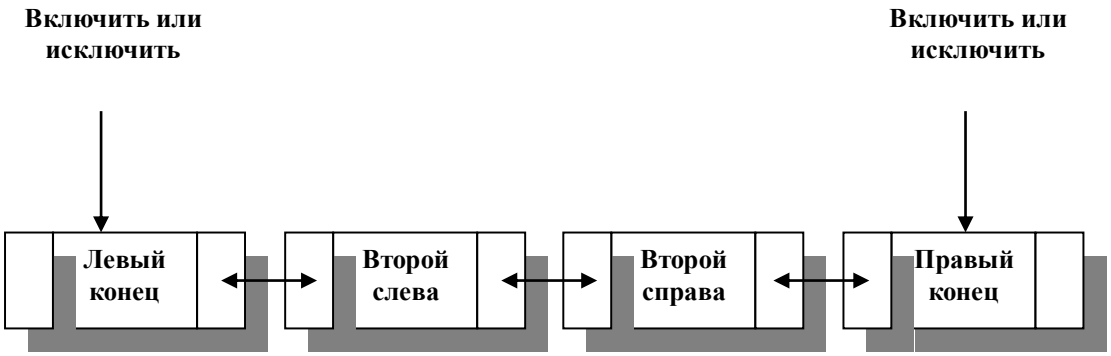
Очередь иногда называют — циклической памятью или списком типа FIFO ("first-in-first-out" — "первым включается — первым исключается"). Другими словами, у очереди есть голова (head) и хвост (tail).

В очереди новый элемент добавляется только с одного конца. Удаление элемента происходит на другом конце. В данном случае это может быть только 4 элемент. Очередь по сути однонаправленный список, только добавление и исключение элементов происходит на концах списка.

Стек (stack) — линейный список, в котором все включения и исключения делаются в одном конце списка. Стек называют пуш-даун (push-down) списком, реверсивной памятью, гнездовой памятью, магазином, списком типа LIFO ("last-in-first-out" — "последним включается — первым исключается"). Стек — часть памяти ОЗУ компьютера, которая предназначается для временного хранения байтов, используемых микропроцессором. Действия со стеком производится при помощи регистра указателя стека. Любое повреждение этой части памяти приводит к фатальному сбою.



Дек (deck) (стек с двумя концами) — линейный список, в котором все включения и исключения делаются на обоих концах списка. Еще один термин "архив" применялся к декам с ограниченным выходом, а деки с ограниченным входом называли "перечнями", или "реестрами".



Рекурсивно можно представить не только алгоритм решения задачи, но и обрабатываемую информацию. Рассмотрим применение рекурсивных процедур при обработке рекурсивных структур данных.

Предварительно обратимся к понятиям списка, указателя и динамической переменной языка Паскаль.

Информационную часть можно описать как INTEGER (целый тип), REAL (действительный), CHAR (символьный) и т.д. Для отображения указателя (горизонтальной стрелки) в языке Паскаль введен особый тип данных, который называется указателем. Для его описания нет ключевого слова, вместо него введен символ ^.

Структуру данных, рассмотренную в виде списка, можно представить на языке Турбо Паскаль следующим образом:

```
TYPE LINK = ^ELEMENT;
```

```
        ELEMENT =RECORD
    INFORM: CHAR;
                                        NEXT: LINK;
END;
```

Здесь LINK - указатель, указывает на ELEMENT. Структура элемента ELEMENT отражена в виде записи, состоящей из двух частей: INFORM и NEXT.

Обратите внимание на характер структуры данных. В начале описания тип данных LINK указывает на ELEMENT у которого, в свою очередь, одна из составляющих NEXT является типом указателя LINK, а LINK указывает на ELEMENT. Получается замкнутый круг. Такая структура данных называется рекурсивной.

В разделе определения типов допускается менять местами описание указателя и описание элемента, например

```
TYPE ELEMENT = RECORD
  INFORM: CHAR;
                                        NEXT: LINK;
END;
LINK = ^ELEMENT;
```

Чтобы иметь возможность использовать в программе переменные типа ELEMENT (то есть переменные, имеющие такую же структуру), необходимо описать их как переменные типа указателя, например

VAR A, B, C: LINK;

Где A, B, C называются переменными-указателями, которые обозначаются в программа со стрелкой ^: A ^, B ^, C ^.

Доступ к элементу записи осуществляется с указанием составного (уточненного) имени, содержащего внутри себя символ точки, например:

A^. INFORM: = ‘Z’;

C^. INFORM: = B^. INFORM;

Каждый тип указателей среди своих возможных значений содержит значение NIL (зарезервированное слово языка Паскаль), которое не указывает ни на один элемент. Чаще всего NIL используется при формировании начала или конца списка, например

A^. LINK: = NIL;

Следует обратить внимание на важный факт: в определении типа данных переменные-указатели A,B , C описаны как указатели (VAR A,B , C : LINK) а не как переменные типа ELEMENT. В этом случае при трансляции память выделяется только для указателей, а для переменных, имеющих структуру записи ELEMENT не выделяется . В языке Паскаль существует понятие динамической переменной, которая начинает существовать в результате вызова стандартной процедуры NEW, например NEW (A). Это означает выделение области памяти и формирование её адреса для создания новой переменной, имеющей структуру записи.

Таким образом, если в программе была объявлена переменная типа указателя, то в результате вызова NEW (A) формируется переменная типа ELEMENT (состоящая из двух частей INFORM и NEXT). Только после этого возможен доступ к элементу записи, например

A^. INFORM: = ‘Z’; или A ^. LINK: = NIL;

Если динамическая переменная становится ненужной, то выделенная область памяти освобождается с помощью стандартной процедуры DISPOSE, например DISPOSE (A).

Задача. Составить программу на языке Паскаль для формирования обхода узлов бинарного дерева.
Набор способа обхода дерева позволяет ввести отношение порядка для узлов дерева.

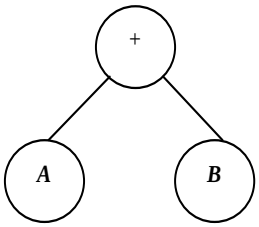
Решение. Для размещения узлов дерева в памяти ПЭВМ воспользуемся списковой структурой данных. Каждый элемент этой структуры соответствует некоторому узлу дерева и описывается записью следующего вида:

```
TREE=RECORD
LEFT: ^TREE;
  RIGHT: ^ TREE;
IDENT: CHAR;
END;
```

где LEFT и RIGHT - указатели (адреса) элементов структуры данных, представляющие узлы, которые являются соответственно корнями левого и правого поддерева данного узла, в поле IDENT находится односимвольный идентификатор узла.

Наиболее распространены три способа обхода узлов дерева, которые получили следующие названия:

- обход в направлении слева направо (обратный порядок, инфиксная запись);
- сверху вниз (прямой порядок, префиксная запись);
- снизу вверх (концевой порядок, постфиксная запись).



КЛП	***Прямой***	***+AB***	***Префиксная***
ЛКП	***Обратный***	***A+B***	***Инфиксная***
ЛПК	***Концевой***	***AB+***	***Постфиксная***

Рис.2. Три различных обхода по бинарному дереву и соответствующие формы записи

В результате обхода дерева, приведенного на рис.3, каждым из трех способов порождаются следующие последовательности прохождения узлов:
слева направо: **A+B*C-D** (обратный порядок);
сверху вниз: ***+AB-CD** (прямой порядок);
снизу вверх: **AB+CD-*** **(**концевой порядок).

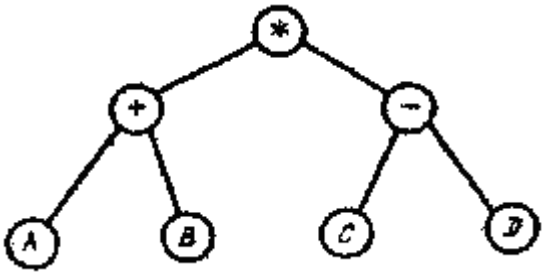


Рис.3. Бинарное дерево

Ниже приведены программы для создания в памяти ПЭВМ списковой структуры бинарного дерева (процедура CREATE) и обхода его узлов в порядке сверху вниз. (процедура PREORDER)

23.1 Формирование бинарного дерева

Листинг 1.

```
TYPE                                    { ОПИСАНИЕ ДЕРЕВА }
NODE   =   ^TREE;                       { ТИП УКАЗАТЕЛЯ УЗЛА ДЕРЕВА              }
TREE   =  RECORD                        { СТРУКТУРА УЗЛА БИНАРНОГО ДЕРЕВА        }
                LEFT  :^TREE;           { УКАЗАТЕЛЬ ЛЕВОГО ПОДДЕРЕВА              }
                RIGHT :^TREE;           { УКАЗАТЕЛЬ ПРАВОГО ПОДДЕРЕВА             }
                IDENT : CHAR;           { ИДЕНТИФИКАТОР УЗЛА ПЕРЕВА               }
        END;
PROCEDURE CREATE(VAR A:NODE);

{       РЕКУРСИВНАЯ ПРОЦЕДУРА СОЗДАНИЯ В ОПЕРАТИВНОЙ ПАМЯТИ
СТРУКТУРЫ БИНАРНОГО ДЕРЕВА    В ПЕРЕМЕННОЙ "A" ФОРМИРУЕТСЯ АДРЕС
КОРНЯ ПОЛУЧЕННОГО ДЕРЕВА ИЛИ ПОДДЕРЕВА }

VAR
        B : NODE;                       { АДРЕС ПОДДЕРЕВА ДАННОГО УЗЛА      }
        R : CHAR;                       { РАБОЧАЯ ПЕРЕМЕННАЯ                }
 BEGIN
        NEW(A);                 { РЕЗЕРВИРОВАНИЕ ПАМЯТИ ДЛЯ НОВОГО УЗЛА ДЕРЕВА
        }
        WRITE(‘ ВВЕДИТЕ ИМЯ УЗЛА> ’);
        READLN(A^.IDENT);
 WRITE(‘ ЕСТЬ ЛЕВОЕ ПОДДЕРЕВО У УЗЛА ‘,A^.IDENT,’  ? (Y/N) ‘);
        READLN(R);
        IF (R='Y')  THEN
        BEGIN
        CREATE(B);              { ФОРМИРОВАНИЕ ЛЕВОГО ПОДДЕРЕВА УЗЛА  }
                A^.LEFT:=B;
        END
        ELSE
        A^.LEFT:=NIL;           { У ДАННОГО УЗЛА НЕТ ЛЕВОГО ПОДДЕРЕВА  }

  WRITE(‘ ЕСТЬ ПРАВОЕ ПОДДЕРЕВО У УЗЛА  ’,A^.IDENT,’ ? (Y/N)  ‘);
        READLN(R);
        IF (R='Y')  THEN
        BEGIN
        CREATE(B);              { ФОРМИРОВАНИЕ ПРАВОГО ПОДДЕРЕВА УЗЛА }
                A^.RIGHT:=B;
        END
        ELSE
       A^.RIGHT:=NIL;           { У ДАННОГО УЗЛА НЕТ ПРАВОГО ПОДДЕРЕВА }
END;
```

23.2 Рекурсивная процедура обхода узлов дерева сверху-вниз

Листинг 2.

```
PROCEDURE   PREORDER(A:NODE);
BEGIN
        IF                A<>NIL                                                THEN
        BEGIN
        WRITE(A^.IDENT:2);                    { ПЕЧАТЬ ИДЕНТИФИКАТОРА УЗЛА }
        PREORDER(A^.LEFT);          { ОБХОД ЛЕВОГО ПОДДЕРЕВА        }
        PREOROER(A^.RIGHT);         { ОБХОД ПРАВОГО ПОДДЕРЕВА       }
        END;
END;
```

В процедуре CREATE функция NEW(A) резервирует в памяти ПЭВМ область для размещения записи типа TREE. В операторе вводится идентификатор текущего узла дерева (один символ) и заносится в поле IDENT записи, адрес которой в данный момент хранятся в переменной А. Далее на экран выдается запрос, есть ли левое поддерево у данного узла. Если в ответ введён символ Y (да), то рекурсивно вызывается процедура CREATE для формирования левого поддерева. После ее завершения в переменной В возвращается адрес узла – корня левого поддерева и он запоминается в поле LEFT текущей записи.

Аналогично формируется правое поддерево. Выход из рекурсии происходит при обработке концевых вершин дерева. В записи, представляющей эти узлы, в поля LEFT и RIGHT заносится константа NIL - неопределенный адрес.

Если проанализировать последовательность прохождения узлов в порядке сверху вниз, то можно установить следующее. Для любого узла, например *, сначала фиксируется факт прохождения через данный узел, затем просматриваются все узлы, входящие в его левое поддерево, а в последнюю очередь просматриваются узлы,
составляющие его правое поддерево. Тогда алгоритм обхода бинарного дерева в порядке сверху вниз имеет следующий вид.

Шаг I. Посетить корень дерева (напечатать его идентификатор),

Шаг 2. Пройти сверху вниз левое поддерево корневого узла.

Шаг 3. Пройти сверху вниз правое поддерево.

Описанный алгоритм реализуется в приведенном примере рекурсивной процедурой. Условие А ≠ NIL позволяет обнаружить концевые вершены дерева и обеспечивает выход из рекурсии.

В основной программе выполняется последовательное обращение к описанным выше подпрограммам создания и обхода бинарного дерева. Заметим, что начальное значение переменной указателя ROOT определяется в процедуре CREATE. Это значение используется для указания адреса корневого узла сформированного бинарного дерева при обращении к процедуре обхода его узлов в порядке сверху вниз.

24 Лабораторное задание

Для проведения лабораторной работы необходимо выполнить следующие действия.

I). Исследовать работу стека, очереди и дека , определив алгоритмы работы указанных структур данных . Результаты записать в тетрадь.

Используя программу **DataStruct** , задать исходное множество элементов, содержащее 12- 15 элементов.

Система работает в диалоговом режиме с использованием “меню”. Вся необходимая информация во время работы системы отображается на экране дисплея и не требует специальных пояснений.

2). сформировать бинарные деревья , содержащие 8, 10 , 12 элементов и выполнить для каждого из них три вида обхода дерева.

3). Составить и отладить программу итеративного и рекурсивного алгоритмов вычисления суммы.

$$\sum_{i=1}^{n} \mathbf{i}$$

аналогично программам вычисления факториала.
Задание выполняется при домашней подготовке. Для определения времени выполнения программы (в секундах) необходимо воспользоваться стандартной встроенной функцией GETTIME .
Сравнить время выполнения этих алгоритмов.
4. Используя программу **TREE_WD** , сформировать бинарные деревья , содержащие 9, 10 ,11 элементов и выполнить для каждого из них балансировку.

25 Требования к отчету

Отчет должен содержать:
1)конспект лабораторной работы;
2)три вида обхода дерева для своего варианта;
3)программу итеративного рекурсивного вычисления суммы;
4)результаты выполнения работы;
5)вывода по работе.

26 Контрольные вопросы

1.Что понимается под рекурсией и итерацией в математике?
2.Каковы особенности итеративного алгоритма?
3.Каковы особенности рекурсивного алгоритма?
4.В чем состоит методика анализа рекурсивного алгоритма?
5.В каких случаях целесообразно использовать рекурсивный или итеративный алгоритм
6.Приведите примеры итерации и рекурсии.
7.Все ли языки программирования дают возможность рекурсивного вызова процедур?
8.Приведите пример рекурсивной структуры данных.
9.Что такое указатели и динамические переменные в языке Паскаль?

27 Литература

1. Т. Кормен, Ч. Лейзерсон, Р. Ривест «Алгоритмы: построение и анализ». М.: МЦНМО, 2000.
2. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
3. Полосухин Б.М., Поддубная Л.М. Рекурсивные функции и алгоритмы. М.: МИЭТ, 1985.
4. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на языке Си. Учеб. пособие. М : Финансы и статистика, 2004.

3 Лабораторная работа - Алгоритмы построения остовного дерева сети

Цель работы: ознакомление с алгоритмами построения остовного дерева графа (сети) и методикой оценки их эффективности.
Продолжительность работы: - 2 часа.

28 Теоретические сведения

Предположим, что необходимо принять решение, связанное с организацией сети компьютеров в различных территориальных пунктах. Это решение является довольно сложным и зависит от большого количества факторов, которые включают в себя вычислительные ресурсы, доступные в каждом пункте, соответствующие уровни потребностей, пиковые нагрузки на систему, возможное неэффективное использование основного ресурса в системе и, кроме того, стоимость предлагаемой сети. В эту стоимость входят приобретение оборудования, прокладка линий связи, обслуживание системы и т.д. Необходимо определить стоимость такой сети.

Нетрудно видеть, что сформулированная здесь задача имеет много других аналогов. Например, требуется соединить несколько населенных пунктов линиями телефонной связи таким образом, чтобы все эти пункты были связаны в сеть, и чтобы стоимость прокладки коммуникаций была минимальной. Вместо телефонных линий можно говорить о прокладке водопроводных коммуникаций, о строительстве дорог и т. д. Решение подобных задач возможно с использованием теории графов (сетей).

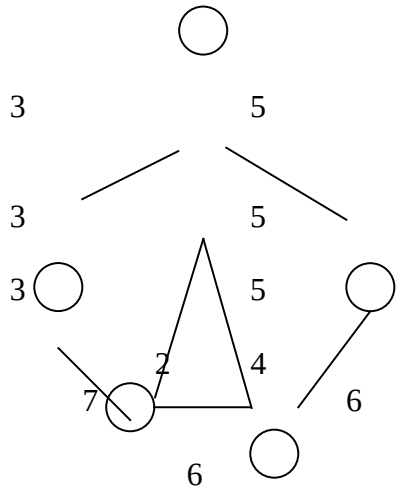
Пусть ***G=(V,E)*** - связный неориентированный граф, содержащий циклы, т.е. замкнутые маршруты, где **V –** множество вершин, а **E** – множество ребер. Остовным (покрывающим) деревом называется подграф, не содержащий циклов, включающий все вершины исходного графа, для которого сумма весов ребер минимальна.

Введем понятие цикломатического числа (**γ**) показывающего, сколько ребер на графе нужно удалить, чтобы в нём не осталось ни одного цикла. Цикломатическое число равно, увеличенной на единицу разности между количеством ребер и количеством вершин графа.

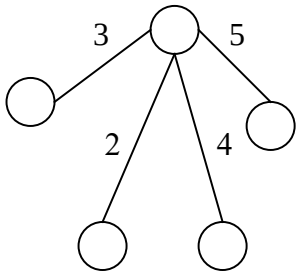
$\gamma = n - m + 1$, где n – количество ребер , m – количество вершин,

Например, для графа, изображенного на рисунке, цикломатическое число равно

$$\gamma = n - m + 1 = 7-5+1=3$$



Это значит , что если на графе убрать три ребра , то в нем не останется ни одного цикла, а суммарный вес ребер равен 14.



29 Алгоритмы Крускала и Прима

Для построения остовного дерева графа используются алгоритмы Крускала и Прима. На рис.1 показана схема алгоритма Крускала.

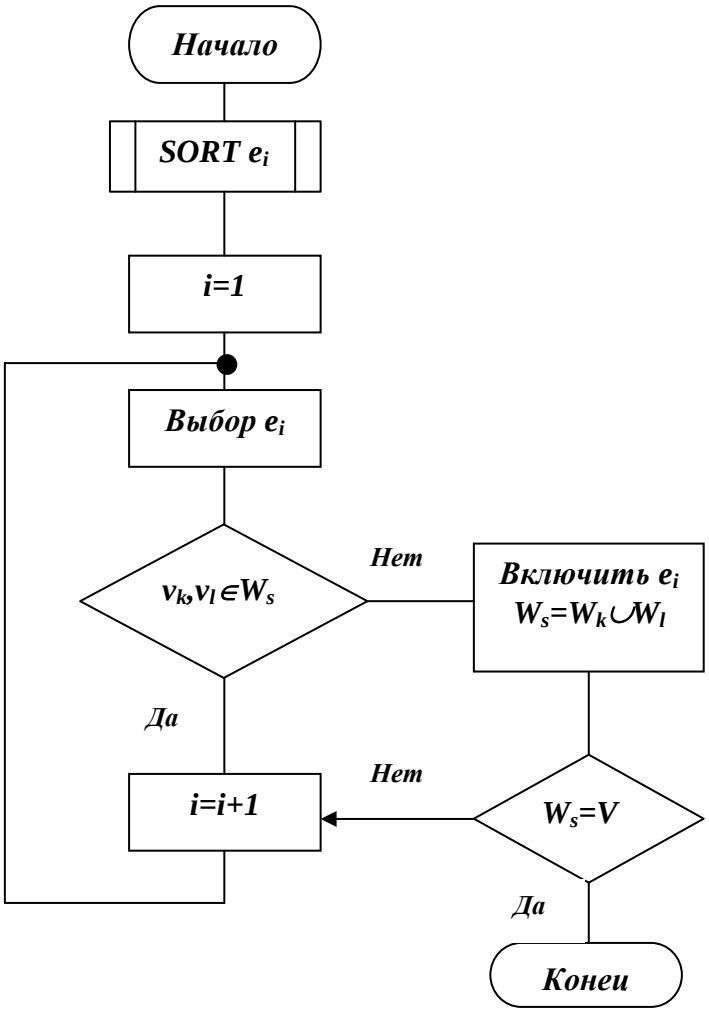


Рис.1. Схема алгоритма Крускала

Блок ***Sort*** e_i предполагает предварительную сортировку весов ребер в порядке их возрастания. В начале работы алгоритма предполагается, что в искомом остове не проведено ни одно ребро (т.е. остов состоит из изолированного множества вершин $v_1, v_2, \ldots v_m$, где m - количество вершин графа). Поэтому считается, что множество W_s имеет вид: $Ws = \{ \{v_1\}, \{v_2\}, \ldots \{v_n\} \}$, где $\{v_i\}$ обозначает множество, состоящее из единственной изолированной вершины v_i. Проверка $v_k, v_l \in W_s$ предполагает установление факта: входят ли вершины v_k, v_l во множество W_s как изолированные, или они сами входят в подмножества постепенно увеличивающихся связных вершин W_k, W_l, каждое из которых имеет вид: $W_k = \{\ldots, v_k, \ldots\}$ и $W_l = \{\ldots, v_l, \ldots\}$.

Если обе вершины v_k, v_l содержатся в одном из подмножеств W_k, W_l, то ребро ***(k,l)*** в остов не включается, в противном случае данное ребро включается в остов, а множества W_k, W_l объединяются. Работа алгоритма Крускала заканчивается, когда множество

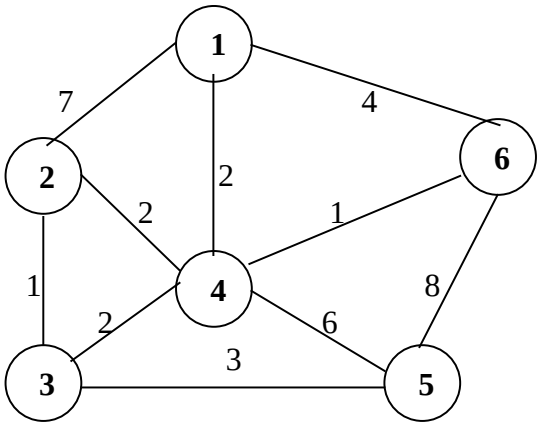
W_s совпадет по мощности с множеством ***V***- множеством всех вершин графа. Нетрудно видеть, что это произойдет, когда все вершины графа окажутся связными.

29.1 Пример со схемой микрорайона

Пусть дана схема микрорайона.

Необходимо соединить дома телефонным кабелем, таким образом, чтобы его длина была минимальной.

Схему микрорайона представим взвешенным графом.



Определим цикломатическое число графа $\gamma = n - m + 1 = 10-6+1=5$,
т.е. на графе необходимо удалить 5 ребер.

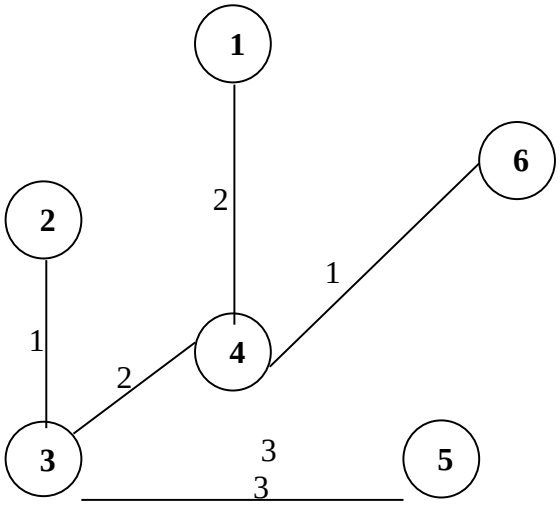
Первоначально каждая вершина исходного графа помещается в одноэлементное подмножество, то есть считаем, что все вершины изолированы, т.е не связаны.

Ребро включается в остовное дерево, если оно связывает вершины, принадлежащие разным подмножествам, при этом вершины объединяются в новое подмножество.

Ребро	Вес	Множества вершин	Операция
		{V1},{V2},{V3},{V4},{V5},{V6}	
{V2,V3}	1	{V2,V3},{V1},{V4},{V5},{V6}	Вкл.
{V4,V6}	1	{V2,V3},{V4,V6},{V1},{V5}	Вкл.
{V1,V4}	2	{V2,V3},{V1,V4,V6},{V5}	Вкл.
{V3,V4}	2	{V1,V2,V3,V4,V6},{V5}	Вкл.
{V2,V4}	2	———————————	Искл.
{V3,V5}	3	{V1,V2,V3,V4,V5,V6}	Вкл.

В таблицу последовательно включаются ребра в порядке возрастания их весов.
Ребро{V2,V3} связывает две вершины, принадлежащие разным подмножествам{V2} и {V3}. Поэтому ребро включается в остовное дерево, а вершины объединяются в одно подмножество{V2,V3}.
Ребро {V4,V6} также связывает вершины из разных подмножеств, оно включается в остовное дерево, а вершины образуют подмножество{V4,V6}.
Вершины V2 и V4 находятся в одном подмножестве, поэтому ребро {V2,V4} исключается из рассмотрения.

Алгоритм заканчивает работу, когда все вершины объединяются в одно множество, при этом оставшиеся ребра не включаются в остовное дерево.
Последовательно просматривая таблицу, получим схему соединения телефонным кабелем домов в микрорайоне.

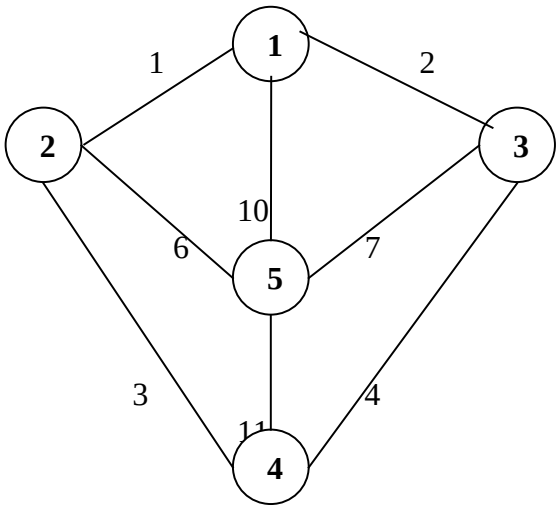


В методе Прима от исходного графа переходим к его представлению в виде матрицы смежности. На графе выбирается ребро минимального веса. Выбранное ребро вместе с вершинами образует первоначальный фрагмент остовного дерева. Затем анализируются длины ребер от каждой вершины фрагмента до оставшихся невыбранных вершин. Выбирается минимальное ребро и присоединяется к первоначальному фрагменту, и т. д.
Процесс продолжается до тех пор, пока в остовное дерево не будут включены все вершины исходного графа.

29.2 Пример со схемой компьютерной сети

Пусть дана схема компьютерной сети.

Необходимо соединить компьютеры таким образом , чтобы длина проводки была минимальной.



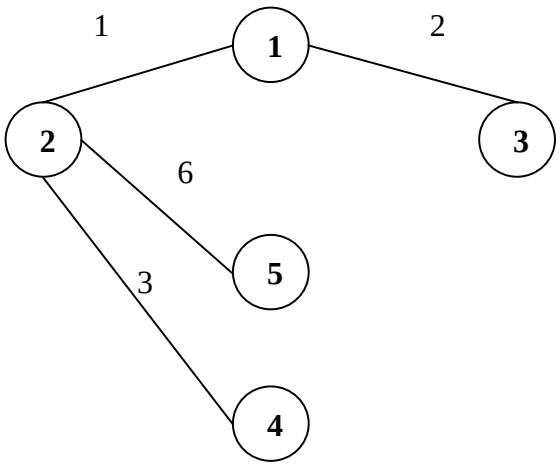
Определим цикломатическое число графа: $\gamma = n - m + 1 = 8 - 5 + 1 = 4$

Выбранные вершины	**Невыбранные вершины**			
	V1	V2	V3	V4
V5	10	**6**	7	11
V2	**1**	*	7	3
V1	*	*	**2**	3
V3	*	*	*	**3**

При просмотре строки таблицы находим минимальную величину веса ребра, выделяем её и больше этот столбец, в котором находится эта величина, в рассмотрении не участвует.

Для построения остовного дерева необходимо просмотреть столбцы таблицы снизу вверх и зафиксировать первое появление минимальной величины.

В итоге получим остовное дерево минимального веса:



30 Лабораторное задание

Для проведения лабораторной работы необходимо выполнить следующие действия.

1. Изучить основные понятия теории графов

2. Вызвать систему **Ostov1**, включающую в себя изучение методов Крускала и Прима.

Система работает в диалоговом режиме с использованием “меню”. Вся необходимая информация во время работы системы отображается на экране дисплея и не требует специальных пояснений.

Для графов, содержащих 10 и 12 вершин построить остовные деревья двумя методами.

По окончании работы выставляется оценка.

3. Вызвать системы **Ostov2 и Ostov3** для исследования методов Крускала и Прима.

Результаты записать в тетрадь.

31 Требования к отчёту

Отчёт должен содержать:

5) конспект лабораторной работы;
6) примеры работы методов Крускала и Прима;
7) результаты выполнения работы;
8) выводы по работе;

<u>Контрольные вопросы</u>

1. Что понимается под остовным деревом?
2. Каковы особенности методов Крускала и Прима?
3. В чём состоит методика анализа сложности алгоритмов построения остовного дерева графа?

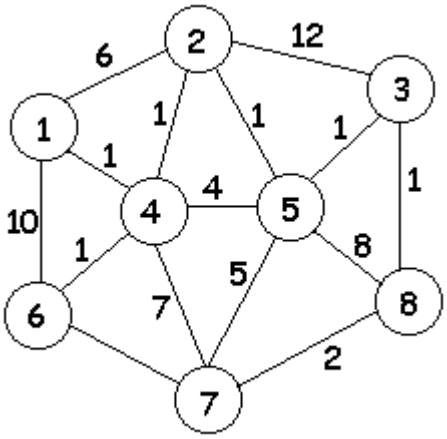
32 Литература

1. Т. Кормен, Ч. Лейзерсон, Р. Ривест. Алгоритмы: построение и анализ. М.: МЦНМО, 2000.
2. Д.Кнут Искусство программирования , том 1. Основные алгоритмы. Уч . пос. М.:Изд. Дом " Вильямс ", 2000.
3. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
4. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на языке Си. Учеб. пособие. М : Финансы и статистика, 2004.

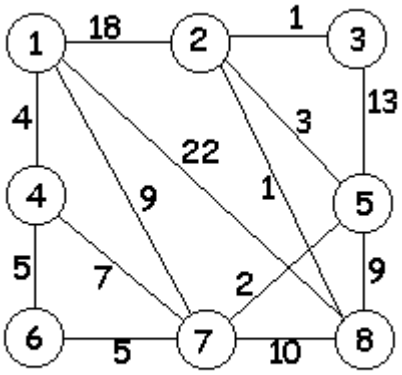
33 Задание к лабораторной работе 4

Построить остовное дерево графа методами Крускала и Прима.

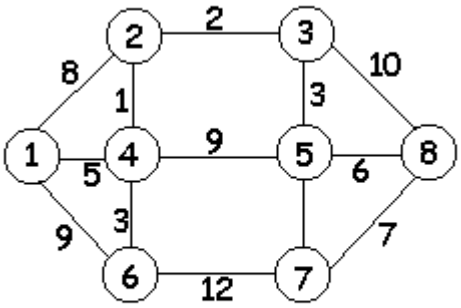
1; 24



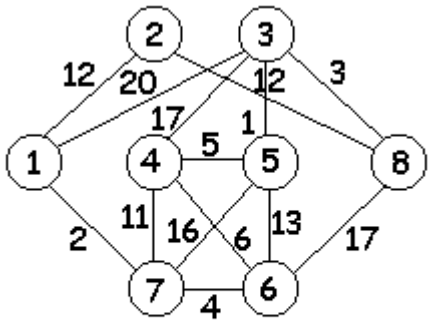
2; 23



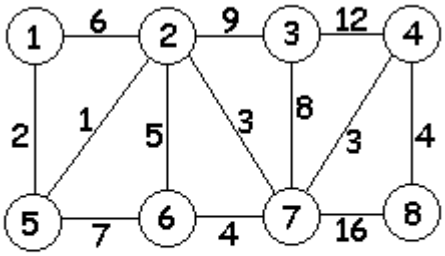
3; 22



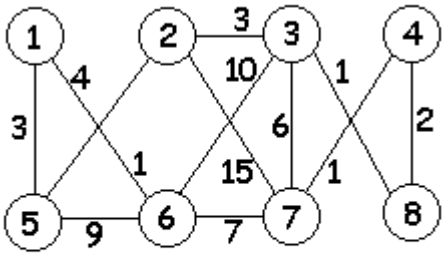
4; 21


2
3
12
20
12
3
17
1
5
1
4
5
8
11
16
6
13
2
17
7
4
6

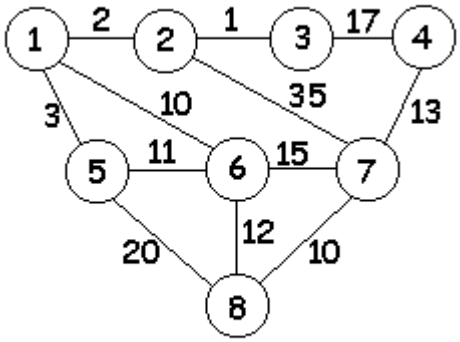
5; 20


1
6
2
9
3
12
4
2
1
5
3
8
3
4
5
7
6
4
7
16
8

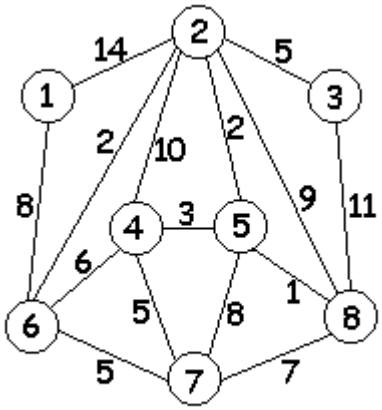
6; 19


1
2
3
3
4
4
10
1
3
6
2
1
15
1
5
9
6
7
7
8

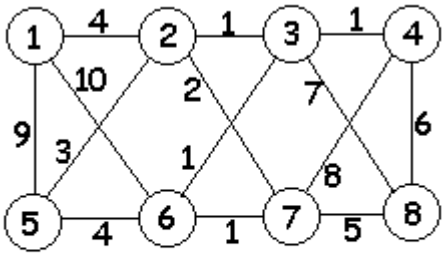
7; 18


1
2
3
4
5
6
7
8
2
1
17
3
10
35
13
11
15
12
20
10

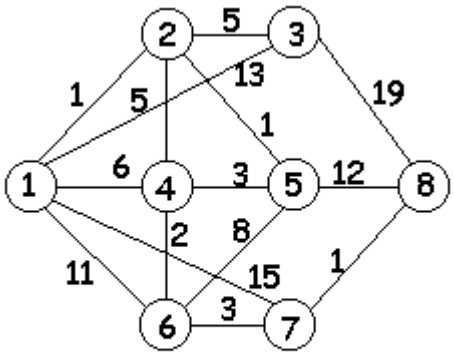
8; 17


1
2
3
4
5
6
7
8
14
5
2
10
2
8
9
11
3
6
5
8
1
5
7

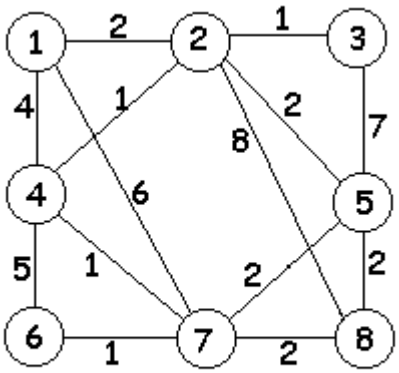
9; 16


1
2
3
4
5
6
7
8
4
1
1
10
2
7
9
3
1
8
6
4
1
5

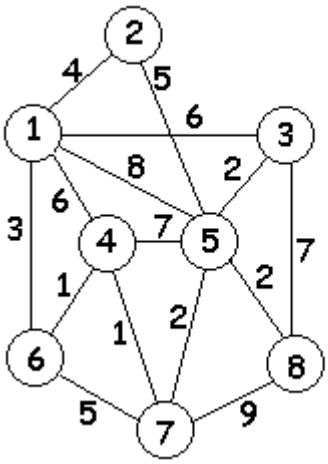
10; 15; 25



11; 14

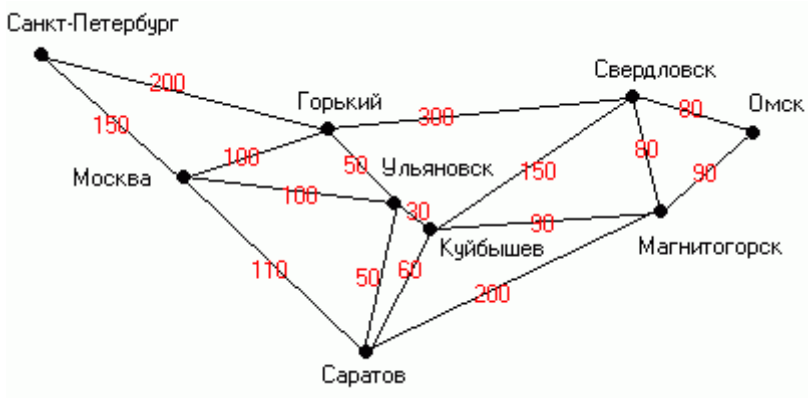


12; 13

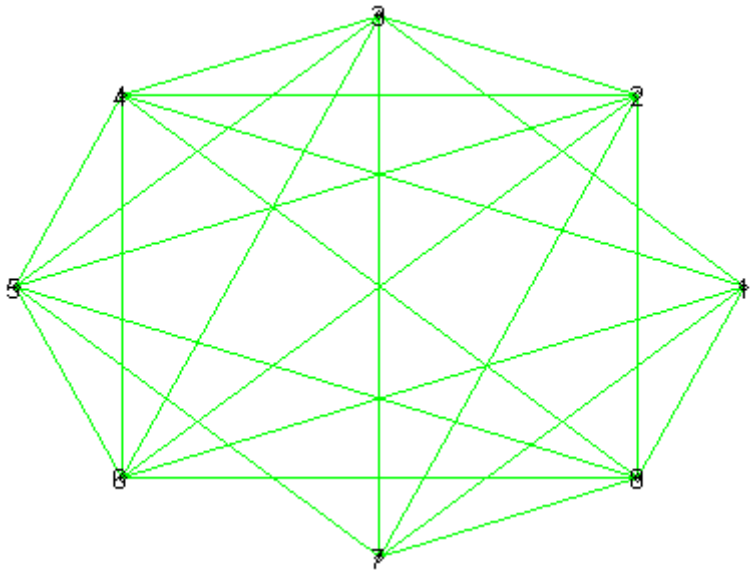


34 Решение задач

Задача 1. Дана карта дорог между городами, где указана длина каждой дороги (данные не совпадают с настоящими). Соединить все города автострадой минимальной длины, т.е. построить остовное дерево методами Прима и Крускала.



Задача 2. Задать веса ребрам и построить остовное дерево графа



Задача 3. Между 9 планетами Солнечной системы введено космическое сообщение. Ракеты летают по следующим маршрутам: Земля-Меркурий, Плутон-Венера, Земля-Плутон, Плутон-Меркурий, Меркурий-Венера, Уран-Нептун, Нептун-Сатурн, Сатурн-Юпитер, Юпитер-Марс и Марс-Уран. Можно ли добраться с Земли до Марса?

Задача 4. Может ли в государстве, в котором из каждого города выходит 3 дороги, быть ровно 100 дорог?

Задача 5. Какое максимальное число висячих вершин может иметь дерево, обладающее 9 вершинами? Какое минимальное число висячих вершин оно может иметь? Сделайте рисунки таких деревьев.

Задача 6. В доме отдыха Вишкиль 57 корпусов. Электрик решил соединить телефонными проводами каждый корпус ровно с пятью другими. Сможет ли он это сделать?

Задача 7. У предприятия имеется 4 завода и 9 торговых точек. Каждый завод должен поставлять свою продукцию во все торговые точки. Сколько всего дорог необходимо проложить, чтобы осуществить задуманное? Определите вид полученного графа.

Задача 8. На участке 3 дома и 3 колодца. От каждого дома к каждому колодцу ведет тропинка. Когда владельцы домов поссорились, они задумали проложить дорогу от каждого дома к каждому колодцу так, чтобы не встречаться на пути к колодцам. Может ли осуществиться их намерение?

Задача 9. Муравей забрался в банку из-под сахара, имеющую форму куба. Сможет ли он последовательно обойти все рёбра куба, не проходя дважды по одному ребру?

Задача 10. На занятии 20 школьников решили каждый по 6 задач, причём каждая задача была решена ровно двумя школьниками. Можно ли организовать разбор всех задач таким образом, чтобы каждый школьник рассказал ровно по 3 задачи.

4 Лабораторная работа - Алгоритмы нахождения на графах кратчайших путей

Цель работы: изучение некоторых алгоритмов нахождения на графах кратчайших путей; исследование эффективности этих алгоритмов.
Продолжительность работы – 2 часа.

35 Теоретические сведения

Задача отыскания в графе (как ориентированном, так и неориентированном) кратчайшего пути имеет многочисленные практические приложения. С решением подобной задачи приходится встречаться в технике связи (например, при телефонизации населенных пунктов), на транспорте (при выборе оптимальных маршрутов доставки грузов), в микроэлектронике (при проектировании топологии микросхем) и т.д.

Путь между вершинами i и j графа считается кратчайшим, если вершины i и j соединены минимальным числом ребер (случай не взвешенного графа) или если сумма весов ребер, соединяющих вершины i и j, минимальна (для взвешенного графа).
В настоящее время известны десятки алгоритмов решения поставленной задачи.

Важным показателем алгоритма является его эффективность. Применительно к поставленной задаче эффективность алгоритма может зависеть в основном от двух параметров графа: число его вершин и число весов его ребер. В данной лабораторной работе для определения кратчайшего расстояния между вершинами графа исследуются два алгоритма: метод динамического программирования и метод Дейкстры.

36 Метод динамического программирования.

Метод рассматривает многостадийные процессы принятия решения. При постановке задачи динамического программирования формируется некоторый критерий. Процесс разбивается на стадии (шаги), в которых принимаются решения, приводящие к достижению общей поставленной цели. Таким образом, метод динамического программирования - метод пошаговой оптимизации.

Введем функцию f_i, определяющую минимальную длину пути из начальной в вершину i . Обозначим через $S_{i\,j}$ длину пути между вершинами i и j , а f_j- наименьшую длину пути между вершиной j и начальной вершиной.

Выбирая в качестве i такую вершину, которая минимизирует сумму ($S_{i\,j}$ + f_j) , получаем уравнение

$$f_i = \min \{S_{i\,j} + f_j\}$$

Трудность решения данного уравнения заключается в том, что неизвестная функция входит в обе части равенства. В такой ситуации приходится прибегать к классическому методу последовательных приближений(итераций), используя рекуррентную формулу:

$$f_i^{(k+1)} = \min \{S_{i\,j} + f_j^{(k)}\} ,$$

где $f_j^{(k)}$ - k -е приближение функции.

Возможен другой подход к решению поставленной задачи с помощью метода стратегий. При движении из начальной точки i в конечную k получается приближение $f_i^{(0)} = S_{ik}$, где S_{ik} - длина пути между точками i и k. Следующее приближение – поиск решения в классе двухзвенных ломаных. Дальнейшие приближения ищутся в классе трехзвенных, четырехзвенных и других ломаных.

37 Пример определения кратчайшего пути №1

Определить кратчайший путь из вершины 1 в вершину 10 для графа, представленного на рис. 1.

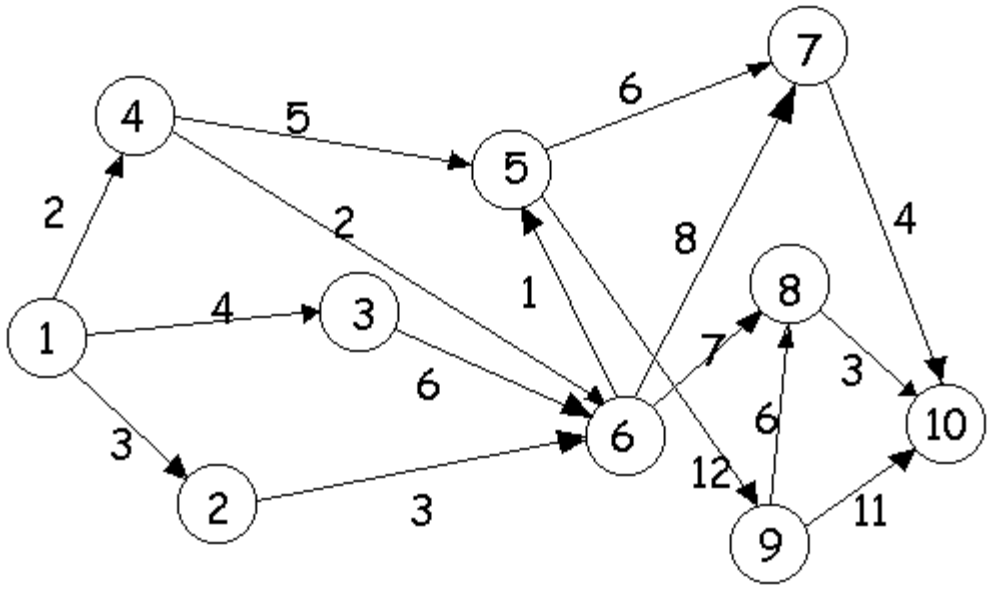


Рис.1. Взвешенный ориентированный граф

Начальные условия: $f_1 = 0$, $S_{11} = 0$.

Находим последовательно значения функции f_i (в условных единицах) для каждой вершины ориентированного графа:

$$f_2 = min\{S_{21} + f_1\} = \{3 + f_1\} = \{3 + 0\} = 3;$$

$$f_3 = min\{S_{31} + f_1\} = \{4 + f_1\} = \{4 + 0\} = 4;$$

$$f_4 = min\{S_{41} + f_1\} = \{2 + f_1\} = \{2 + 0\} = 2;$$

$$f_6 = min \begin{Bmatrix} S_{64} + f_4 \\ S_{63} + f_3 \\ S_{62} + f_2 \end{Bmatrix} = min \begin{Bmatrix} 2 + 2 \\ 6 + 4 \\ 3 + 3 \end{Bmatrix} = 4;$$

$$f_5 = min \begin{Bmatrix} S_{54} + f_4 \\ S_{56} + f_6 \end{Bmatrix} = min \begin{Bmatrix} 5 + 2 \\ 1 + 4 \end{Bmatrix} = 5;$$

$$S_{75} + f_5 \qquad f_7 = min \begin{Bmatrix} 6 + 5 \\ 8 + 4 \end{Bmatrix} = min \begin{Bmatrix} \\ \\ \end{Bmatrix} = 11;$$

$$S_{55} + f_6$$

$$f_9 = min \ \{S_{85} + f_5\} = min = \{12 + 5\} = 17;$$

$$S_{86} + f_6 \qquad f_8 = min \begin{Bmatrix} 7 + 4 \\ S_{89} + f_9 \end{Bmatrix} = min \begin{Bmatrix} \\ 6+17 \\ \end{Bmatrix} = 11;$$

$$f_{10} = min \begin{Bmatrix} S_{10,7} + f_4 \\ S_{10,8} + f_8 \\ S_{10,9} + f_{10} \end{Bmatrix} = min \begin{Bmatrix} 4 + 11 \\ 3 + 11 \\ 11 + 17 \end{Bmatrix} = 14.$$

Длина кратчайшего пути составляет 14 условных единиц. Для выбора оптимальной траектории движения следует осуществить просмотр функций f_i в обратном порядке, то есть с f_{10}. Пусть $f_i = f_{10}$. В данном случае

$$f_{10} = min \begin{Bmatrix} 4 + f_1 \\ 3 + f_8 \\ \\ \end{Bmatrix} = min \begin{Bmatrix} 4 + 11 \\ 3 + 11 \\ \\ \end{Bmatrix} = 14.$$

$11 + f_9$ $\qquad\qquad$ $11 + 17$

Получаем, что $(3 + f_8) = 14$, то есть $fj = f_8$. Значит, из вершины 10 следует перейти к вершине 8. Имеем $fi = f_8$. Рассмотрим функцию

$$f_8 = \min \begin{cases} 7 + f_6 \\ 6 + f_9 \end{cases} = \min \begin{cases} 7 + 4 \\ 6 + 17 \end{cases} = 11,$$

то есть $fj = f_6$ и т.д.

Таким образом, получаем кратчайший путь от вершины 1 к вершине 10:

(1, 4, 6, 8, 10)

Рассмотренный метод определения кратчайшего пути может быть распространен и на неориентированные графы.

38 Пример нахождения кратчайшего пути при условии, что граф неориентированный№2

Для графа (рис. 1) найти кратчайший путь от вершины 1 до вершины 10, рассматривая граф как неориентированный. Матрица смежности весов графа в этом случае имеет вид:

	1	2	3	4	5	6	7	8	9	10
1	**3**	**4**	**2**	**-**	**-**	**-**	**-**	**-**	**-**	**-**
2	**3**	**-**	**-**	**-**	**-**	**3**	**-**	**-**	**-**	**-**
3	**4**	**-**	**-**	**-**	**-**	**6**	**-**	**-**	**-**	**-**
4	**2**	**-**	**-**	**-**	**5**	**2**	**-**	**-**	**-**	**-**
5	**-**	**-**	**-**	**5**	**-**	**1**	**6**	**-**	**12**	**-**
6	**-**	**3**	**6**	**2**	**1**	**-**	**8**	**7**	**-**	**-**
7	**-**	**-**	**-**	**-**	**6**	**8**	**-**	**-**	**-**	**4**
8	**-**	**-**	**-**	**-**	**-**	**7**	**-**	**-**	**6**	**3**
9	**-**	**-**	**-**	**-**	**12**	**-**	**-**	**6**	**-**	**11**
10	**-**	**-**	**-**	**-**	**-**	**-**	**4**	**3**	**11**	**-**

Записываем выражения для функции f_i

$$\begin{cases} f_1 + 3 \\ f_6 + 3 \end{cases} \quad f1 = 0; \quad f2 = \min; \qquad \begin{cases} f_1 + 4 \\ f_6 + 6 \end{cases} \quad f3 = \min; \quad f4 = \min \qquad \begin{cases} f_1 + 2 \\ f_6 + 2 \end{cases} \quad f5 + 5$$

$$\begin{cases} f_4 + 5 \end{cases} \qquad \begin{cases} f_2 + 3 \\ f_3 + 6 \end{cases}$$

$$f_5 = \min \begin{matrix} f_6 + 1 \\ f_9 + 12 \\ f_7 + 6 \end{matrix} ; \quad f_6 = \min \begin{matrix} f_4 + 2 \\ f_5 + 1 \\ f_7 + 8 \\ f_8 + 7 \end{matrix} ; \quad f_7 = \min \begin{cases} f_5 + 6 \\ f_6 + 8 \end{cases}$$

$$f_8 = \min \begin{cases} f_6 + 7 \\ f_9 + 6 \end{cases} ; \quad f_9 = \min \begin{cases} f_5 + 12 \\ f_8 + 6 \end{cases} ; \quad f_{10} = \min \begin{cases} f_7 + 4 \\ f_8 + 3 \\ f_9 + 11 \end{cases} .$$

Указанные целевые функции , представляют собой систему линейных алгебраических уравнений (в примере имеется 10 уравнений и 10 неизвестных).
Рассмотрим один из вариантов ее решения, учитывая, что $f_1 = 0$.
Тогда имеем:

$$2 \quad f_2 = 3; \quad f_3 = 4; \quad f_4 = \min \begin{cases} f_5 + 5 \\ f_6 + 2 \end{cases} = 2;$$

$$f_5 = \min \begin{cases} 7 \\ f_6 + 1 \\ f_9 + 12 \\ f_7 + 6 \end{cases} = \min \begin{cases} 7 \\ f_6 + 1 \end{cases} ; \quad f_6 = \min \begin{cases} 6 \\ 10 \\ 4 \\ f_5 + 1 \\ f_7 + 1 \\ f_8 + 7 \end{cases} = \min \begin{cases} 4 \\ f_5 + 1 \end{cases} = 4$$

Подставив выражение для f_6 в f_5, получим

$$7 \quad f_5 = \min \begin{cases} 4 + 1 \\ f_5 = \min \begin{cases} 4 + 1 \\ f_5 + 1 + 1 \end{cases} \end{cases} = 5. \quad = 5.$$

Тогда

$$f_7 = 11; \quad f_8 = \min \begin{cases} 11 \\ f_9 + 6 \end{cases} ; \quad f_9 = \min \begin{cases} 17 \\ f_8 + 6 \end{cases} ; \quad f_{10} = \min \begin{cases} 15 \\ f_8 + 3 \\ f_9 + 11 \end{cases} .$$

Подставляя f_9 в f_8, получаем

$$f_8 = \min \begin{cases} 17 + 6 \\ f_8 + 6 + 6 \end{cases} = 11. \qquad 11$$

Окончательно имеем: $f_9 = 17$; $f_{10} = 14$.

39 Метод Дейкстры

Алгоритм Дейкстры предназначен для нахождения кратчайшего пути между вершинами в неориентированном графе. Идея алгоритма следующая, сначала выберем путь до начальной вершины равным нулю, и заносим эту вершину во множество уже выбранных, расстояние от которых до оставшихся невыбранных вершин определено. На каждом следующем этапе находим следующую невыбранную вершину, расстояние до которой наименьшее и соединённую ребром с какой-нибудь вершиной из множества выбранных (это расстояние будет равно расстоянию до уже выбранной вершины плюс длина ребра).

Найти кратчайший путь на графе от вершины L до вершины D (рис.2).

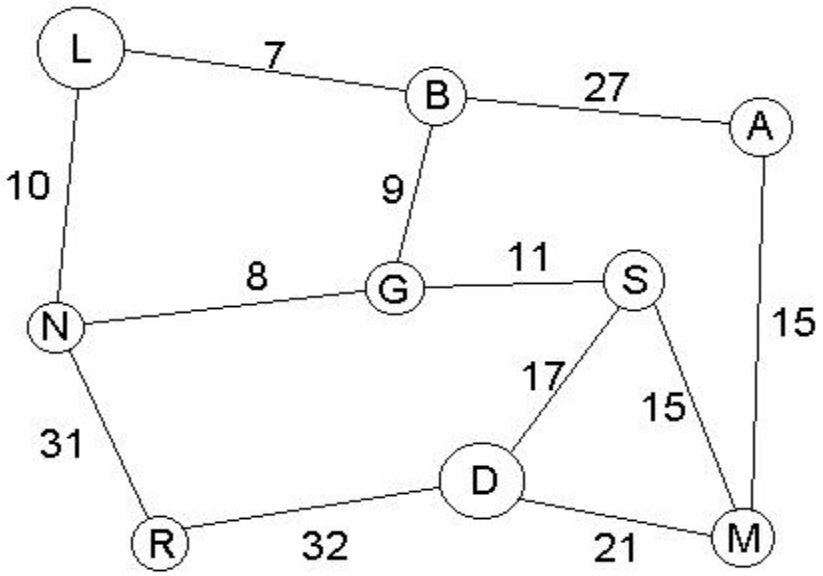


Рис.2. Взвешенный неориентированный граф

Запишем алгоритм в виде последовательности шагов.

Шаг 1. Определяются расстояния от начальной вершины L до всех остальных .

Длина пути до выбранной вершины	Выбранная вершина	Невыбранные вершины							
		B	G	N	R	D	S	M	A
0	L	7	∞	10	∞	∞	∞	∞	∞

7	B

Шаг 2. Выбираем наименьшее расстояние от L до B, найденная вершина B принимается за вновь выбранную. Найденное наименьшее расстояние добавляется к длинам ребер от новой вершины B до всех остальных. Выбирается минимальное расстояние от B до N. Новая вершина N принимается за выбранную.

Длина пути до выбранной вершины	Выбранная вершина	Невыбранные вершины							
		B	G	N	R	D	S	M	A
0	L	7	∞	10	∞	∞	∞	∞	∞
7	B	∞	16	10	∞	∞	∞	∞	34
10	N								

Для наглядности в дальнейшем вместо знака ∞ будем ставить знак “-“.
Шаг 3. Определяются расстояния от вершины N до всех оставшихся (за исключением L и B) .

Длина пути до выбранной вершины	Выбранная вершина	Невыбранные вершины							
		B	G	N	R	D	S	M	A
0	L	7	∞	10	∞	∞	∞	∞	∞
7	B	∞	16	10	∞	∞	∞	∞	34
10	N	∞	16	∞	41	∞	∞	∞	34
16	G								

Расстояние от вершины L через вершину N до вершины G равно 18. Это расстояние больше, чем расстояние LB+BG= 16, поэтому оно не учитывается в дальнейшем.
Продолжая аналогичные построения, построим таблицу. Таким образом, найдена длина кратчайшего пути между вершинами L и D (44 условных единицы).
Траектория пути определяется следующим образом.
Осуществляем обратный просмотр от конечной вершины к начальной.
Просматриваем столбец, соответствующий вершине, снизу вверх и фиксируем первое появление минимальной величины. В столбце, соответствующем вершине D, впервые минимальная длина 44 появилась снизу в четвертой строке. В этой строке указана вершина S, к которой следует перейти, то есть следующим нужно рассматривать столбец, соответствующий вершине S.

Длина пути до выбранной вершины	Выбранная вершина	Невыбранные вершины							
		B	G	N	R	D	S	M	A
0	L	7	-	10	-	-	-	-	-
7	B	-	16	10	-	-	-	-	34
10	N	-	16	-	41	-	-	-	34
16	G	-	-	-	41	-	16+11= =27	-	34

27	S	-	-	-	41	27+17= =44	-	27+15= =42	34
34	A	-	-	-	41	44	-	42 [34+15]	-
41	R	-	-	-	-	44 [41+32]	-	42	-
42	M	-	-	-	-	44 [42+21]	-	-	-
						Min			

В этом столбце минимальная длина, равная 27, указывает следующую вершину G, к которой следует перейти, и т.д. Таким образом, получаем траекторию пути:

(L, B, G, S, D).

40 Алгоритм Флойда

Пусть в матрице A[i,j] записаны длины ребер графа (элемент A[i,j] равен весу ребра, соединяющего вершины с номерами i и j, если же такого ребра нет, то в соответствующем элементе записано некоторое очень большое число). Построим новые матрицы $C_k[i,j]$ (k=0,…,N). Элемент матрицы $C_k[i,j]$ будет равен минимально возможной длине такого пути из i в j, что в качестве промежуточных вершин в этом пути используются вершины с номерами от 1 до k. То есть рассматриваются пути, которые могут проходить через вершины с номерами от 1 до k , но заведомо не проходят через вершины с номерами от k+1 до N. В матрицу записывается длина кратчайшего из таких путей. Если таких путей не существует, записывается то же большое число, которым обозначается отсутствие ребра.

Если вычислена матрица $C_{k-1}[i,j]$, то элементы матрицы $C_k[i,j]$ можно вычислить по следующей формуле: $C_k[i,j]:=\min(C_{k-1}[i,j], C_{k-1}[i,k]+C_{k-1}[k,j])$. В самом деле, рассмотрим кратчайший путь из вершины i в вершину j, который в качестве промежуточных вершин использует только вершины с номерами от 1 до k. Тогда возможно два случая:

Этот путь не проходит через вершину с номером k. Тогда его промежуточные вершины — это вершины с номерами от 1 до k–1. Но тогда длина этого пути уже вычислена в элементе $C_{k-1}[i,j]$.

Этот путь проходит через вершину с номером k. Но тогда его можно разбить на две части: сначала из вершины i доходим оптимальным образом до вершины k, используя в качестве промежуточных вершины с номерами от 1 до k–1 (длина такого оптимального пути вычислена в $C_{k-1}[i,k]$), а потом от вершины k идем в вершину j опять же оптимальным способом, и опять же используя в качестве промежуточных вершин только вершины с номерами от 1 до k ($C_{k-1}[k,j]$).

Выбирая из этих двух вариантов минимальный, получаем $C_k[i,j]$.

Последовательно вычисляя матрицы C_0, C_1, C_2 и т.д. получим искомую матрицу C_N кратчайших расстояний между всеми парами вершин в графе.

Алгоритм Флойда можно свести к последовательности шагов

Присвоить $c_{ij}=0$ для всех i=1,2,...,n и $c_{ij}=\infty$, если в графе отсутствует дуга (x_i x_j).

Присвоение начальных значений.

Шаг 1. присвоить k=0;

Шаг 2. k=k+1.

Шаг 3. Для всех i<>k, таких, что $c_{ik}<>\infty$, и для всех j<>k, таких, что $c_{ik}<>\infty$,

$c_{ij}= [\min[c_{ij}, (c_{ik}+ c_{kj})]$. (9)

Проверка на окончание.

Шаг 4. Если k=n, то матрица $[c_{ij}]$ дает длины всех кратчайших путей. Остановка. Иначе перейти к шагу 2.

41 *Алгоритм Йена*

Пусть граф задан матрицей A[i,j] . Вершина s выбрана как начальная. Найдём длины k наименьших путей до каждой вершины от вершины s (рёбра в одном пути могут повторяться неоднократно), при условии, что эти пути существуют. Результат будет храниться в матрице B, размером N*k, где N-количество вершин. Элемент массива B[i,j] равен j-му по длине пути до вершины i.

Для каждой вершины в массиве C будем хранить количество уже найденных до неё наименьших путей. Изначально все элементы массива C равны нулю. Все элементы матрицы B делаем равными, какой-нибудь большой константе, заведомо большей всех возможных путей. Во время исполнения алгоритма в матрице B будем хранить лучшие k путей до каждой вершины, найденные во время исполнения, при этом первые C[i] путей для вершины i найдены уже окончательно (элементы матрицы B, B[i,1], B[i,2], …, B[i,k] для всех i, упорядочены в возрастающем порядке).

Следовательно, можно действовать следующим образом: пусть уже найдены какие-то кратчайшие пути, тогда чтобы найти следующий по длине, попробуем удлинить каждый из уже полученных на одно ребро. Найдём наикратчайший из них, при чём оканчивающийся на вершину, до которой найдено менее k путей. Его можно вносить в таблицу результата.
Алгоритм Йена можно свести к последовательности шагов

Присвоение начальных значений.

Шаг 1. Найти P^1. Присвоить k=2. Если существует только один кратчайший путь P^1, включить его в список L_0 и перейти к шагу 2.

Если таких путей несколько, но меньше, чем К, включить один из них в список L_c, а остальные в список L_1.
Перейти к шагу 2.

Если существует К или более кратчайших путей P^1, то задача решена. Остановка.

Нахождение отклонений.

Шаг 2. Найти все отклонения P_i^k(k-1)-го кратчайшего пути P^{k-1} для всех i=1, 2, …, q_{k-1}, выполняя для каждого i шаги с 3-го по 6-й.

Шаг 3. Проверить, совпадает ли подпуть, образованный первыми i вершинами любого из P^j путей (j=1, 2,…,k-1). Если да, то присвоить $c(x_i^{k-1}, x_{i+1}^j)=\infty$; иначе ничего не изменять. (При выполнении алгоритма вершина x_1 обозначается s).

Шаг 4. Найти кратчайшие пути S_i^k от x_i^{k-1} к t, исключая из рассмотрения вершины s, x_i^{k-1}, x_3^{k-1},..., x_i^{k-1}. Если существует несколько кратчайших путей, то взять в качестве S_i^k один из них.

Шаг 5. Построить P_i^k, соединяя R_i^k(=s, x^{-1}, x^{-1},…, x^{-1}) с S и поместить P в список L_1.

Шаг 6. Заменить элементы матрицы весов, измененные на шаге их первоначальными значениями и возвратиться к шагу 3.

Выбор кратчайших отклонений.

Шаг 7. Найти кратчайший путь в списке L_1. Обозначить этот путь P^k и переместить его из L_1 в L_0.

Если k=K, то остановка. Список L_0-список К-кратчайших путей.

Если k<K, то присвоить k=k+1, перейти к шагу 2.
Если в L_1 имеется более чем один кратчайший путь (h-путей), то поместить в L_0 любой из них и продолжать как выше до тех пор, пока увеличенное на h число путей, уже находящихся в L_1, не совпадет с К или не превысит его. Тогда остановка.

42 Алгоритм Беллмана- Форда

Пусть в матрице A[i,j] записаны длины ребер графа . Найдём наикратчайшие расстояния от заданной вершины s до всех остальных вершин графа. Алгоритм Беллмана-Форда решает эту задачу при наличии рёбер отрицательного веса. Обозначим через МинСт(s,v,к) наименьшую стоимость проезда из s в v менее чем с k пересадками.

МинСт(s,v,k+1)=min(МинСт(s,v,k),МинСт(s,i,k)+A[i][v](i=1..n))

Искомым ответом является МинСт(s,i,n) для всех i=1..n.

Чтобы найти не только длины наименьших путей до всех вершин, но и сами эти пути используем следующий приём. Параллельно с вычислением массива x будем вычислять матрицу D[i,j]. Если между вершинами s и j существует путь, то в элементе массива D [j,0] будет хранится количество вершин в этом пути, а цепочка вершин, составленная из элементов с D[j,1] по D[j,D[j,0]], будет этим самым путём. Путь до вершины s содержит единственную вершину (D[s,0]=1, D[s,1]=s). Для вычисления матрицы D потребуется немного дополнить текст процедуры:

```
k:= 1;
for i := 1 to n do begin x[i] := a[s][i]; end;
{инвариант: x[i] := МинСт(s,i,k)}     Обозначим через МинСт(s,i,к) наименьшую
   стоимость   проезда из s в i менее чем с k пересадками. Тогда выполняется такое
   соотношение:
for i := 1 to n do begin D[i,0]:=2; D[i,1]:=s; D[i,2]:=i; End;
D[s,0]:=1;

while k <> n do begin
  for i := 1 to n do begin
    for j := 1 to n do begin
     if x[i] > x[j]+a[j][i] then begin
        x[i] := x[j]+a[j][i];
        D[i]:=D[j];
        D[i,0]:=D[j,0]+1;
            D[i,D[i,0]]:=i;
      end;
    end
    {x[i] = МинСт(s,i,k+1)}
  end;
  k := k + 1;
end;
```

43 Литература

1. Т. Кормен, Ч. Лейзерсон, Р. Ривест
   Алгоритмы: построение и анализ. М.: МЦНМО, 2000.
2. Д.Кнут Искусство программирования , том 1. Основные алгоритмы. Уч . пос. М.:Изд. Дом " Вильямс ", 2000.
3. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
4. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на языке Си. Учеб. пособие. М : Финансы и статистика, 2004.
5. А. Ахо, Дж.Хопкрофт, Дж.Ульман, Структуры данных и алгоритмы М: СПб: Киев: Вильямс, 2001г.

44 *Лабораторное задание.*

Для выполнения лабораторной работы необходимо:

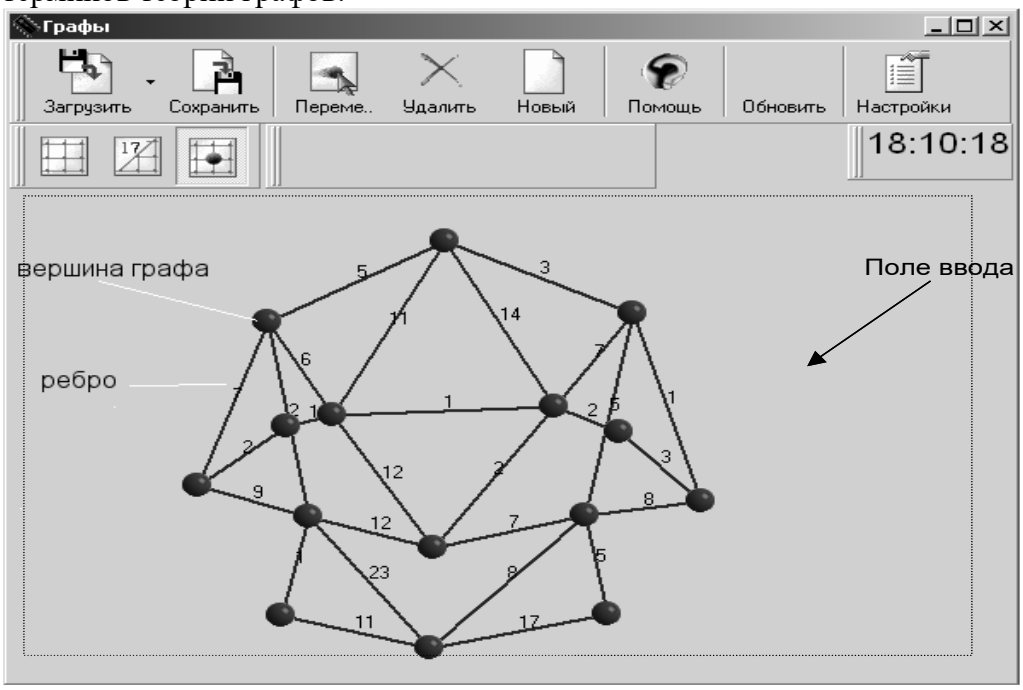
1) Найти кратчайший путь методом динамического программирования для трех графов . Зафиксировать параметры каждого графа(число вершин и ребер), значение пути и время решения задачи (ориентированные графы);

2) Найти кратчайший путь методом Дейкстры для трех графов. Зафиксировать параметры каждого графа (число вершин и ребер), значение пути и время решения задачи.

Системы работает в диалоговом режиме с использованием «меню». Вся необходимая поясняющая информация отображается во время работы системы на экране монитора.

3) Изучить алгоритмы работы с графовыми структурами

Данная программа предназначена для демонстрации работы алгоритмов и облегчения понимания терминов теории графов.



Программа позволяет вводить, редактировать, сохранять графы в файл, загружать из файла. Также реализованы алгоритмы построения Эйлерова цикла, нахождения кратчайшего пути, нахождение пути, содержащего наименьшее количество вершин, решение задачи о раскраске вершин, задачи Коммивояжера.

Для ввода вершины щелкните левой кнопкой мыши в свободное от кнопок поле. Для ввода ребра щелкните правой кнопкой мыши на одной вершине, затем на другой. Для ввода длины ребра щелкните левой кнопкой мыши на введенном ребре. При перемещении вершины нажмите кнопку «Переместить», щелкните левой кнопкой мыши на перемещаемую вершину, затем на свободное место, куда вы хотите переместить вершину, и нажмите кнопку «Переместить». Для удаления вершины нажмите на кнопку «Удалить», затем щелкните

левой кнопкой мыши на вершинах, которые хотите удалить, затем еще раз на кнопку «Удалить».

При включенной кнопке «Выравнивать по сетке» новые вершины будут автоматически выравниваться по координатной сетке.

Третья панель служит для запуска алгоритмов программы. Первая кнопка запускает алгоритм решения задачи Коммивояжера. При нажатии на вторую кнопку программа раскрасит граф минимальным количеством цветов. Если выбрать две различные вершины (щелчком левой кнопки мыши) и нажать на третью кнопку, то программа найдет путь с наименьшим количеством вершин, а если четвёртую, то найдет кратчайший путь между вершинами. При нажатии на пятую кнопку построится Эйлеров цикл.

4) Ознакомиться с алгоритмами Флойда, Йена, Беллмана- Форда

45 Требования к отчету

Отчет должен содержать:

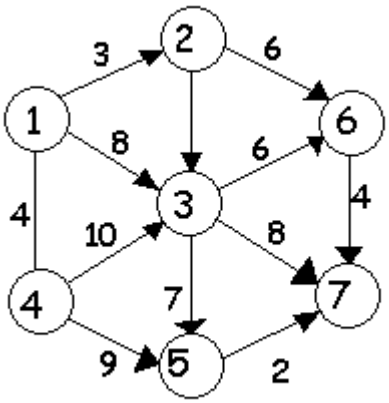
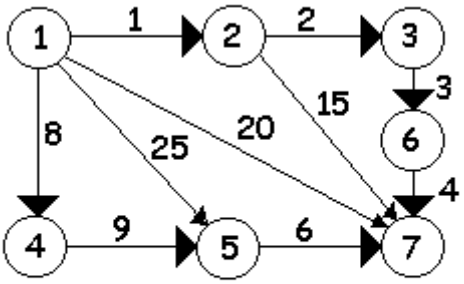
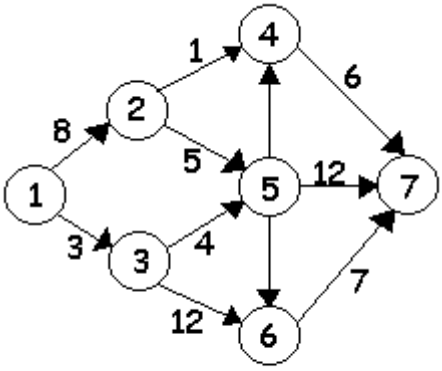
1) Конспект лабораторной работы;
2) Схемы алгоритмов определения в графе кратчайшего расстояния между заданными вершинами для метода динамического программирования и метода Дейкстры;
3) Результаты выполнения работы;
4) Выводы по работе.

<u>Контрольные вопросы</u>

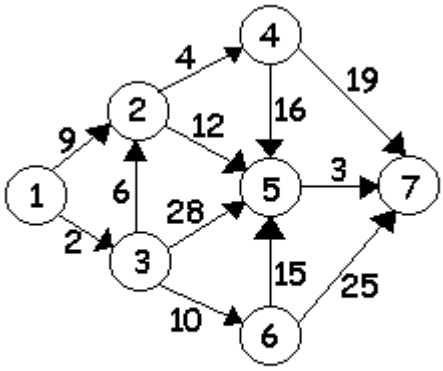
1) Какова теоретическая сложность алгоритмов, рассмотренных в данной работе?
2) В решении каких прикладных задач используется алгоритмы определение в графе кратчайших расстояний между заданными вершинами?
3) Может ли быть применен рассмотренный в работе алгоритм Дейкстры к определению кратчайшего расстояния в ориентированном графе?
4) Как работает алгоритм Дейкстры?
5) Как работает алгоритм динамического программирования применительно к задаче определения в графе кратчайших расстояний между вершинами?

46 Варианты заданий

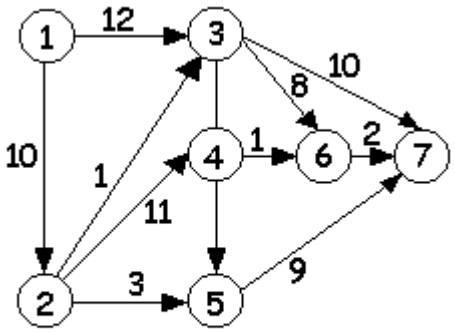
46.1 Задание 1: Найти кратчайший путь на графе между тремя парами вершин методом динамического программирования

Варианты	Граф
1, 13	
2, 14	
2, 14 3, 15	

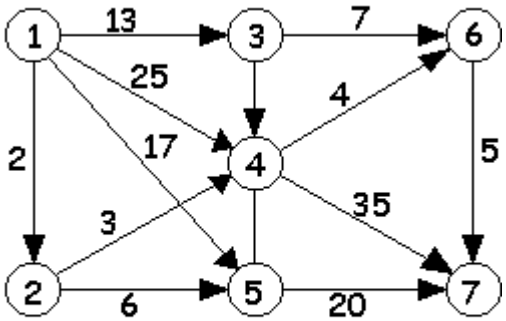
4, 16



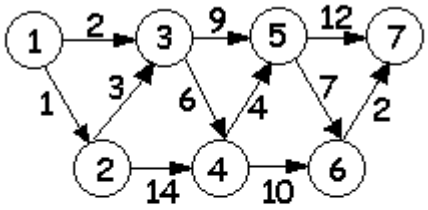
5, 17, 25



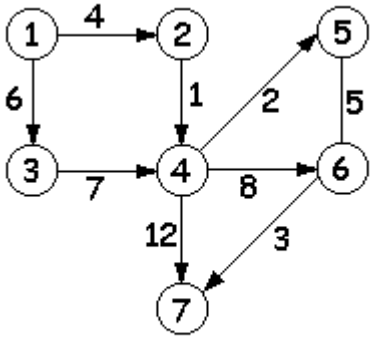
6, 18



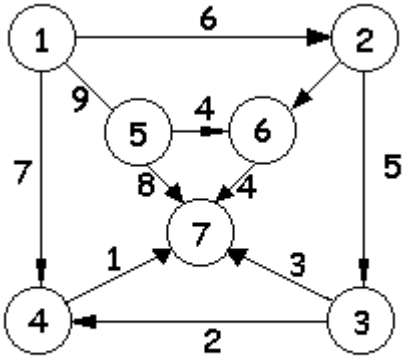
7, 19


1
2
3
4
5
6
7
2
9
12
1
3
6
4
7
2
14
10

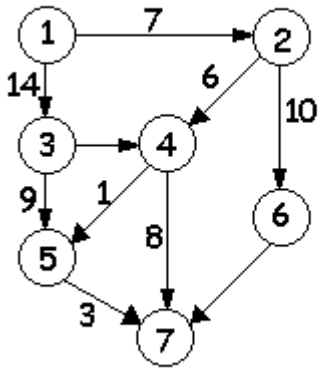
8, 20


1
2
3
4
5
6
7
4
6
1
2
5
7
8
12
3

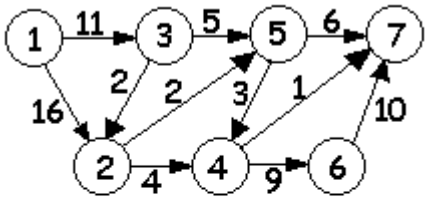
9; 21


1
2
3
4
5
6
7
6
9
4
7
5
8
4
1
3
2

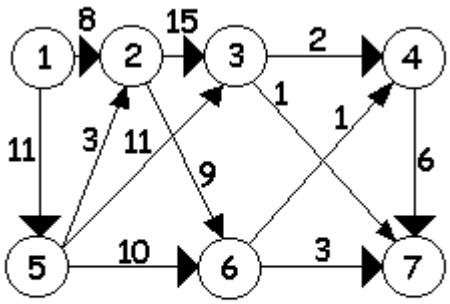
10; 22



11; 23

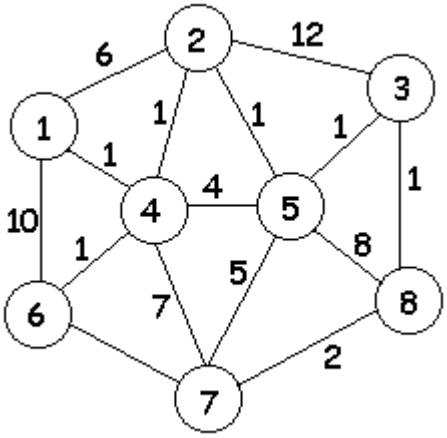


12; 24

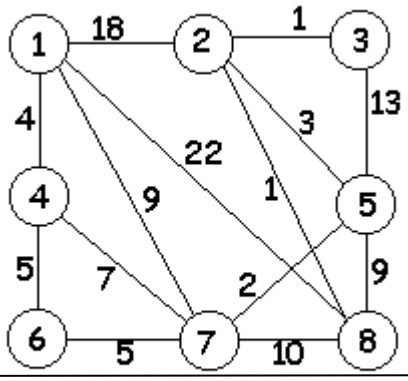


46.2 Задание 2: Найти кратчайший путь между тремя парами вершин методом Дейкстры

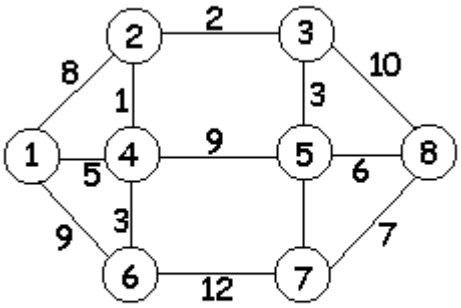
1; 24



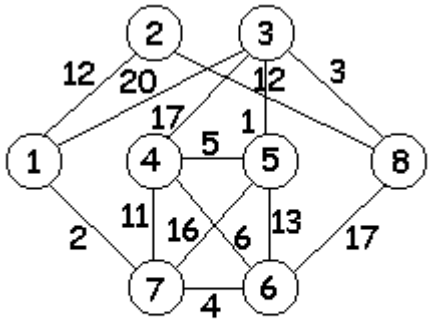
2; 23



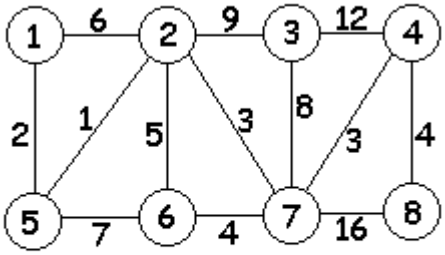
3; 22



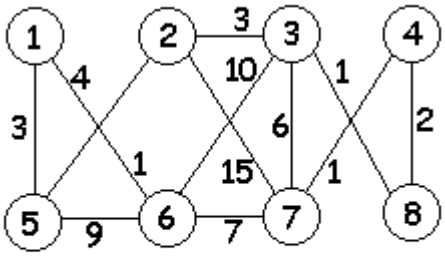
4; 21


2
3
12
20
12
3
17
1
1
4
5
5
8
11
16
6
13
2
17
7
4
6

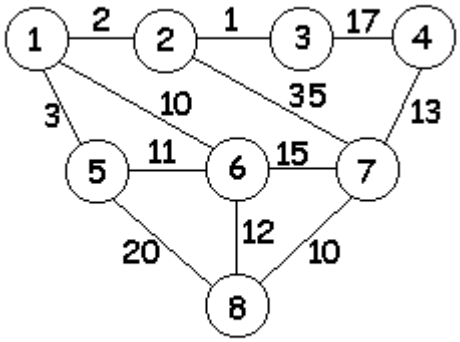
5; 20


1
6
2
9
3
12
4
2
1
5
3
8
3
4
5
7
6
4
7
16
8

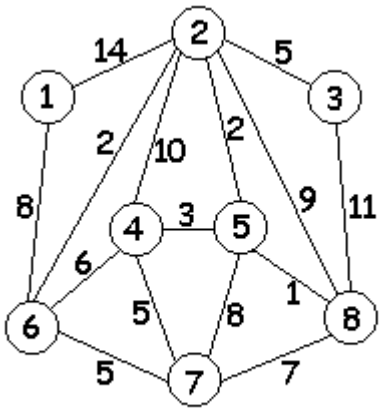
6; 19


1
2
3
3
4
4
10
1
3
6
2
1
15
1
5
9
6
7
7
8

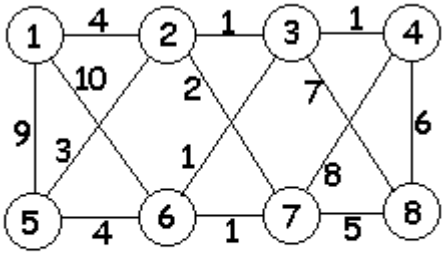
7; 18


1
2
3
4
5
6
7
8
2
1
17
3
10
35
13
11
15
12
20
10

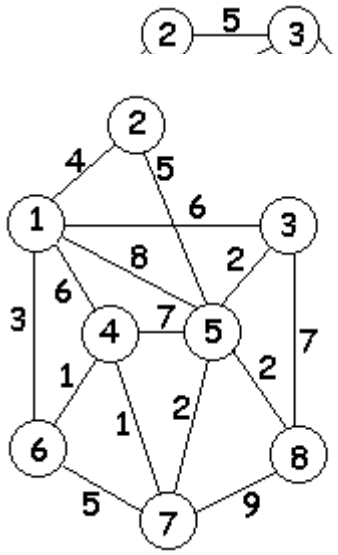
8; 17


1
2
3
4
5
6
7
8
14
5
2
10
2
8
9
11
3
6
5
8
1
5
7

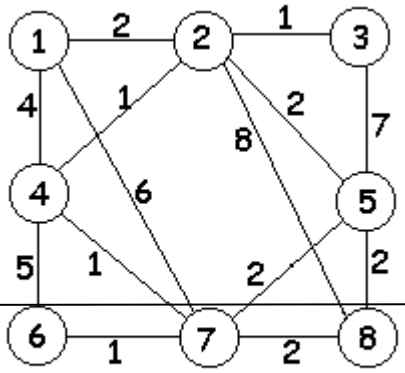
9; 16


1
2
3
4
5
6
7
8
4
1
1
10
2
7
9
3
1
8
6
4
1
5

10; 15; 25



11; 14



12; 13

47 Решение задач

Задание 1. Дана карта автомобильных дорог. Найти кратчайший путь, учитывая длины дорог, из Ярославля до Новосибирска. Использовать только те города, которые отмечены флажком. Совпадет ли кратчайший маршрут, если не брать во внимание длины дорог?

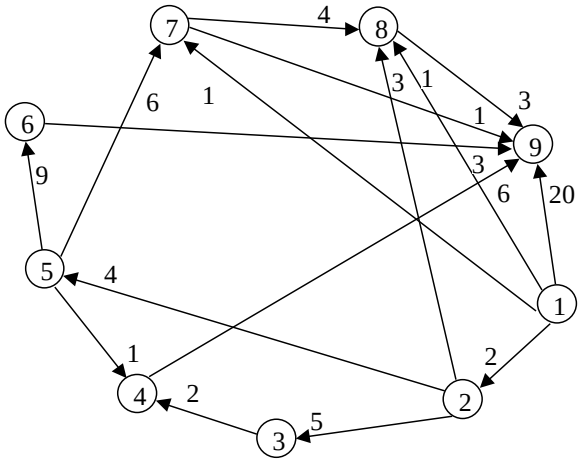
Задание 2. Дана карта Европы. Турагенству необходимо найти кратчайший туристический маршрут между произвольно выбранными странами Европы.

Задание 4. На участке 3 дома и 3 колодца. От каждого дома к каждому колодцу ведет тропинка. Когда владельцы домов поссорились, они задумали проложить дорогу от каждого дома к каждому колодцу так, чтобы не встречаться на пути к колодцам. Может ли осуществиться их намерение?

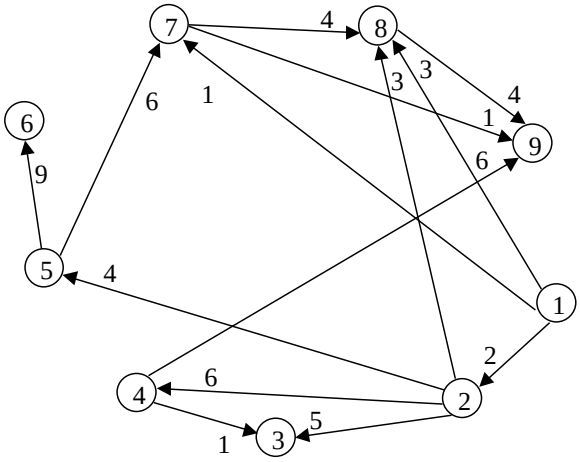
48 Задание на разработку программы

Разработка программу для нахождения кратчайшего пути

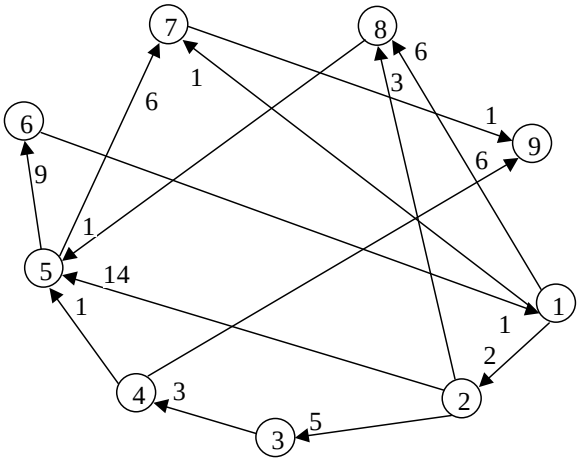
Вариант 1. Путь из 1 в 9



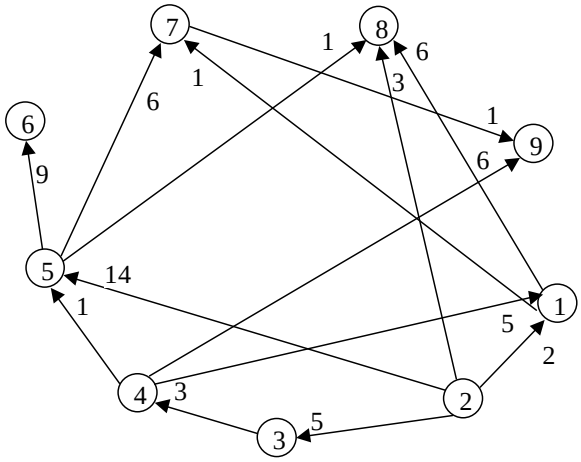
Вариант 3. Путь из 1 в 9



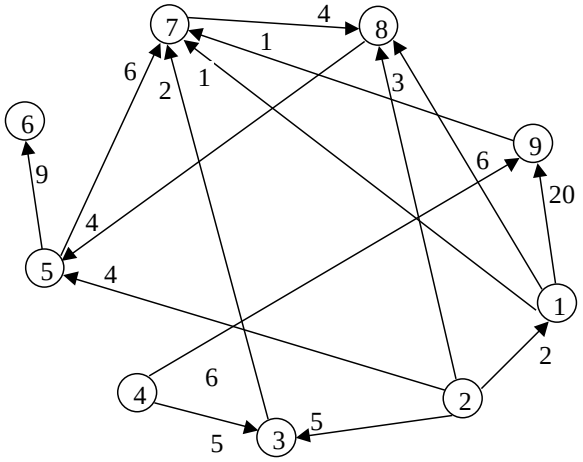
Вариант 2. Путь из 2 в 5



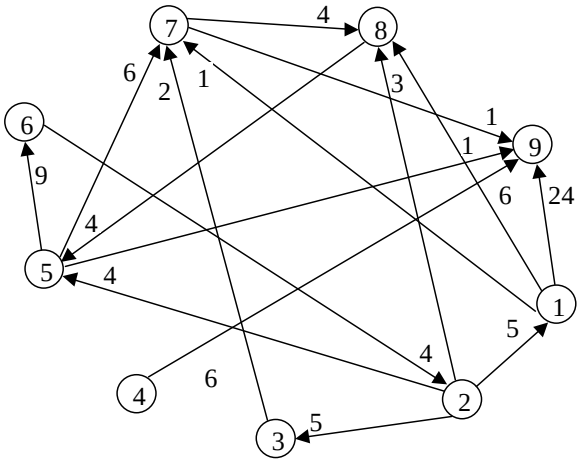
Вариант 4. Путь из 2 в 5



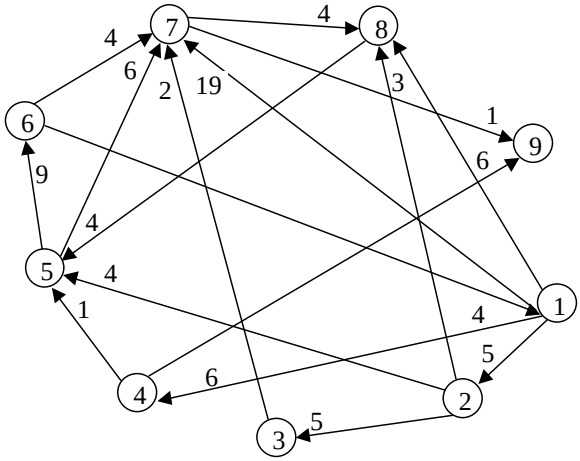
Вариант 5. Путь из 2 в 7



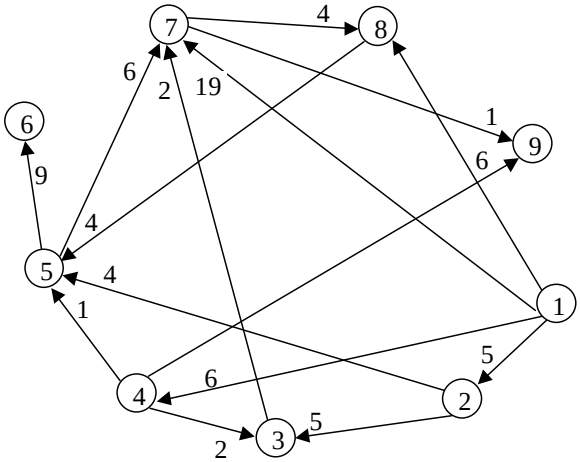
Вариант 8. Путь из 2 в 9



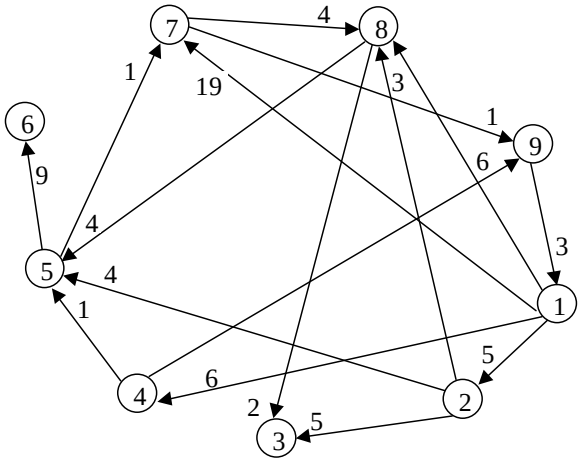
Вариант 6. Путь из 1 в 7



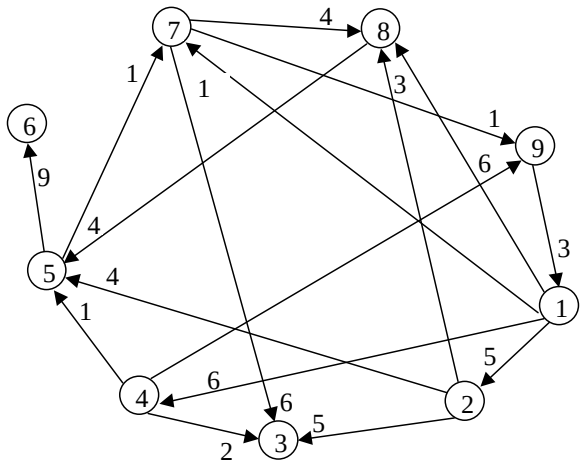
Вариант 9. Путь из 1 в 5



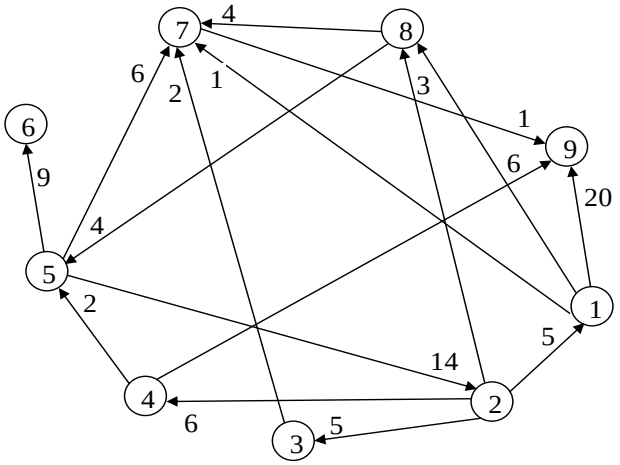
Вариант 7. Путь из 4 в 1



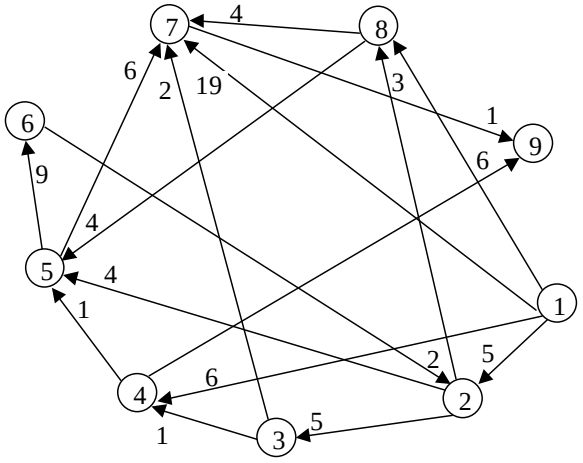
Вариант 10. Путь из 1 в 3



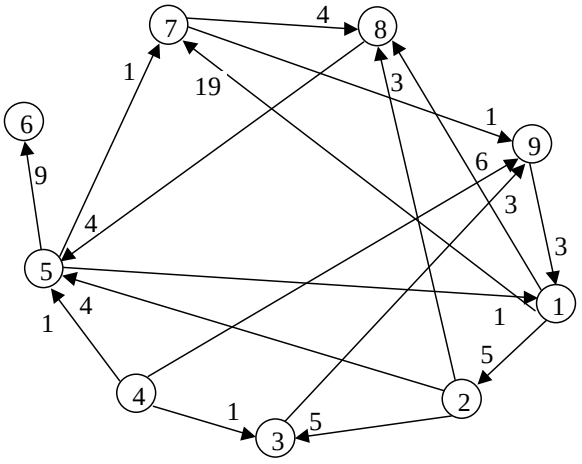
Вариант 11. Путь из 2 в 7



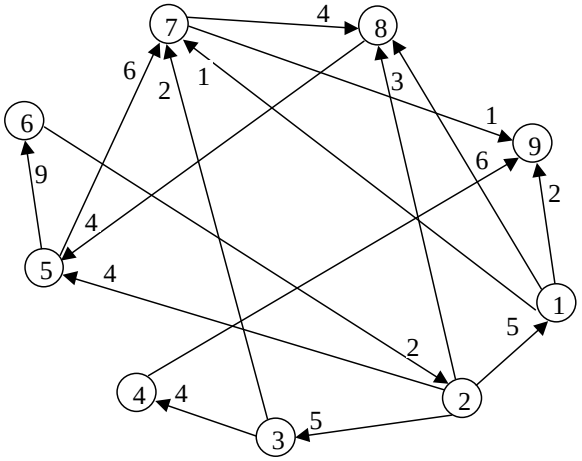
Вариант 12. Путь из 1 в 7



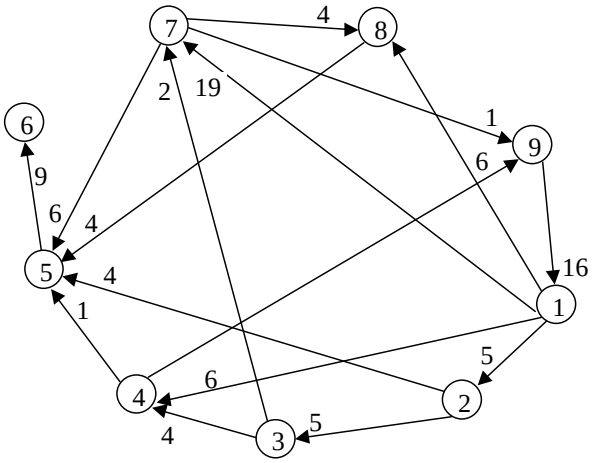
Вариант 13. Путь из 4 в 1



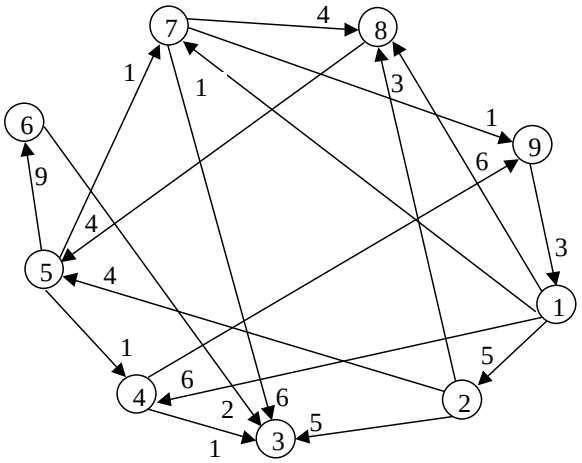
Вариант 14. Путь из 2 в 9



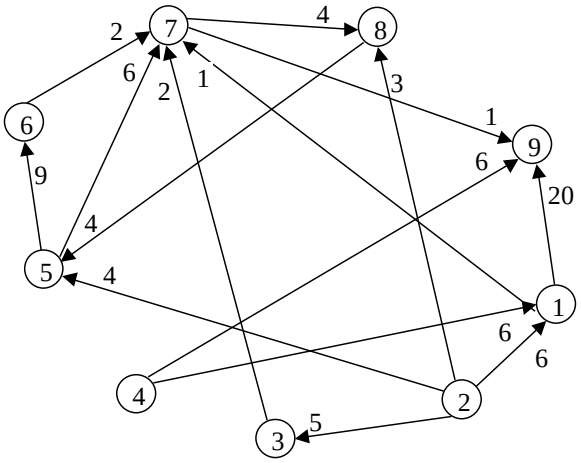
Вариант 15. Путь из 1 в 5



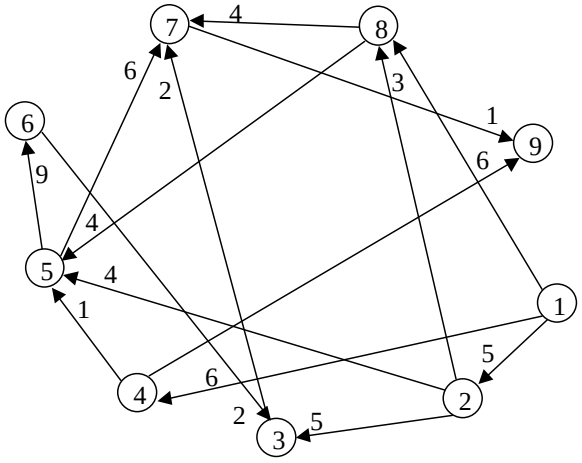
Вариант 16. Путь из 1 в 3



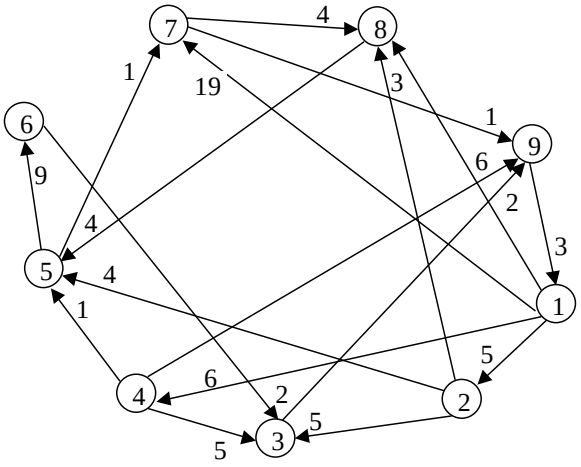
Вариант 17. Путь из 2 в 7



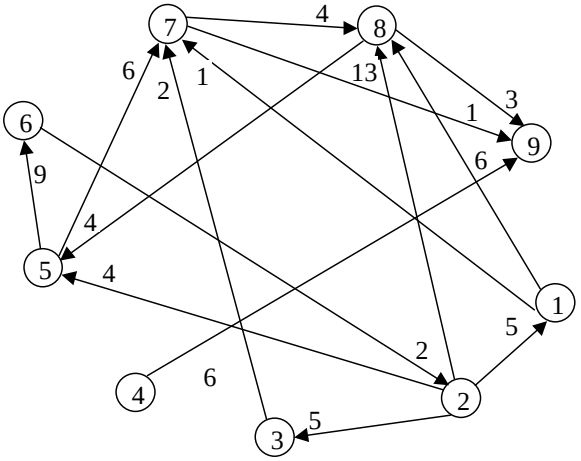
Вариант 18. Путь из 1 в 7



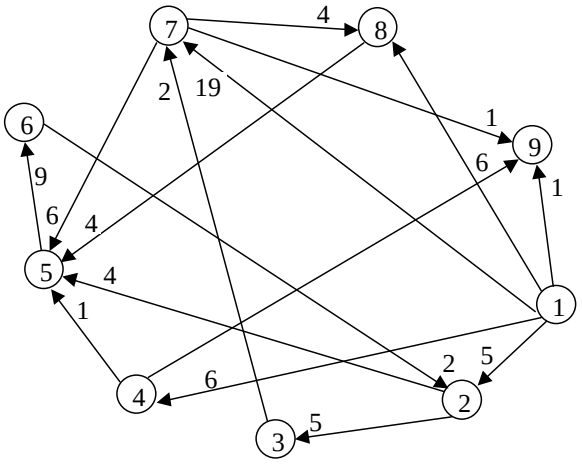
Вариант 19. Путь из 4 в 1



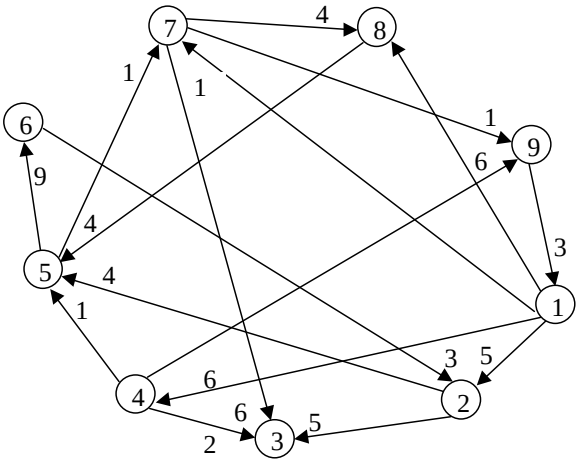
Вариант 20. Путь из 2 в 9



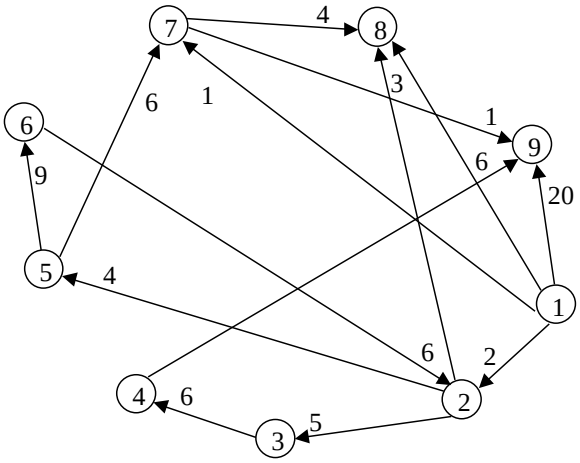
Вариант 21. Путь из 1 в 5



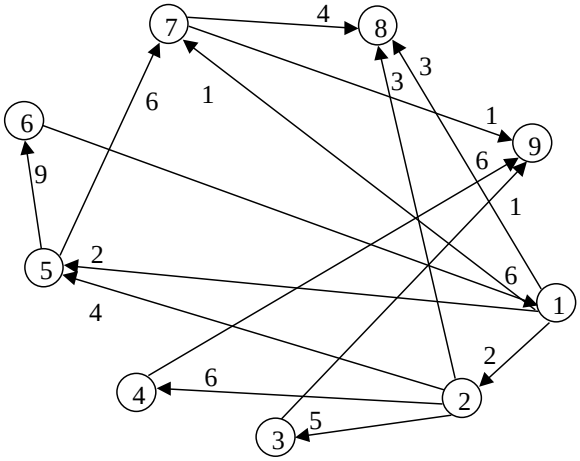
Вариант 22. Путь из 1 в 3



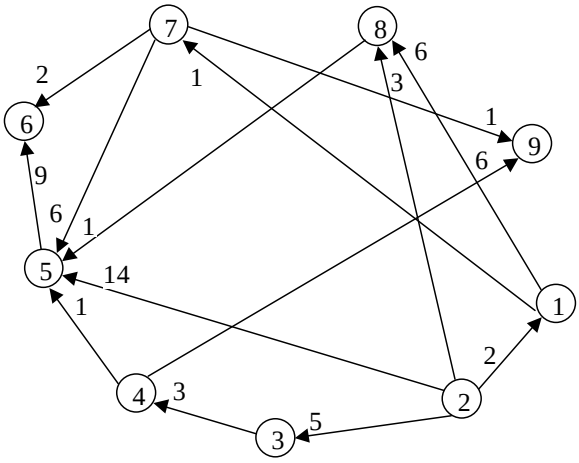
Вариант 23. Путь из 1 в 9



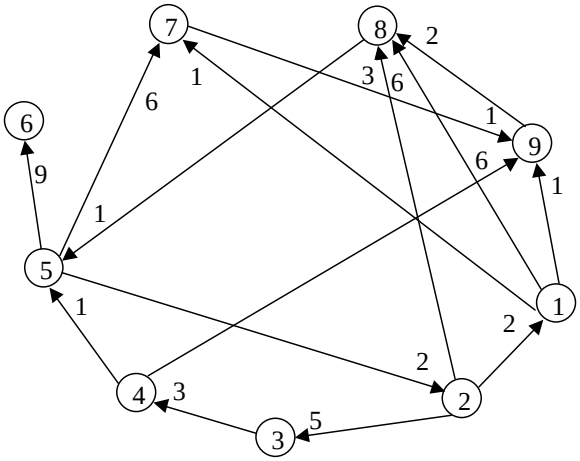
Вариант 25. Путь из 1 в 9



Вариант 24. Путь из 2 в 5



Вариант 26. Путь из 2 в 5



5 Лабораторная работа -Эвристические алгоритмы

Цель работы: ознакомление с эвристическими алгоритмами и методикой оценки их эффективности .

Продолжительность работы: - 2 часа.

49 Теоретические сведения

Эвристический алгоритм – это алгоритм , в котором на определенном этапе используется интуиция разработчика . Если решение , принятое разработчиком , окажется неверным результат все равно будет получен , но за большее число шагов . Таким образом в эвристических алгоритмах можно увеличить скорость получения правильного результата
К эвристическим алгоритмам относятся : **волновой** , **двухлучевой** , **четырехлучевой** , **маршрутный, алгоритмы составления расписания.**

50 Волновой алгоритм

Волновой алгоритм или алгоритм Ли первоначально использовался для поиска пути в лабиринте или в игровых задачах. В настоящее время алгоритм Ли (волновой) является основным в микроэлектронике для трассировки (соединения) элементов интегральных схем. Особенность алгоритма состоит в следующем:

- В лабиринте(на подложке) выбираются две точки А(начальная) и В(конечная). Из начальной точки в четырех направлениях выходит волна. Цифрами обозначается номер фронта волны или ее путевые координаты .

	1	
1	**A**	**1**
	1	

- Путевые координаты определяют шаг распространения волны. Каждый элемент первого фронта волны является источником вторичной волны.

		2		
	2	**1**	**2**	
2	**1**	**A**	**1**	**2**
	2	**1**	**2**	
		2		

Элементы второго фронта генерируют третий фронт и т.д. От запрещенных элементов волна не распространяется.

Процесс продолжается до тех пор, пока не будет достигнут конечный элемент. Траектория пути определяется обратным просмотром, от конечного к начальному. При этом разработчик задает приоритеты движения :

Вверх , Вниз , Влево , Вправо.

От того в каком порядке заданы приоритеты зависит скорость решения задачи.

При построении траектории используется два принципа:

1) Движение осуществляется строго по заданным приоритетам.
2) При построении трассы, т.е. траектории движения, значения путевых координат должны уменьшаться.

Пример 1. Пусть задан лабиринт, где запрещенные элементы заштрихованы.
Найти путь между элементами А и В.

На первом этапе от элемента А распространяется волна до тех пор пока она не достигнет конечного элемента В. На втором этапе выбираются приоритеты движения от конечной точки В к начальной А. Приоритеты выбираются исходя из взаимного расположения начального и конечного элемента. Предположим , что выбраны парадоксальные (логически неверные) приоритеты : вверх , вправо , вниз , влево. В этом случае трасса все равно будет построена, но за большее число шагов (сравнений).

6		10	9	8		10			
5				7		9	10		
4			5	6		8	9	В	
3	2		4	5		7	8		10
	1	2	3	4		6			9
1	А	1	2	3	4	5	6	7	8
2	1		3				7		
3	2		4	5	6		8	9	10

51 Двухлучевой алгоритм.

В двухлучевом алгоритме из начального и конечного элементов одновременно выходят по два луча, трасса считается проведенной, если пересекаются два разноименных луча (от разных источников). Если на пути луча встречается запрещенный элемент, его обход осуществляется по второму приоритетному направлению, характерному для лучей выходящих из одной точки.

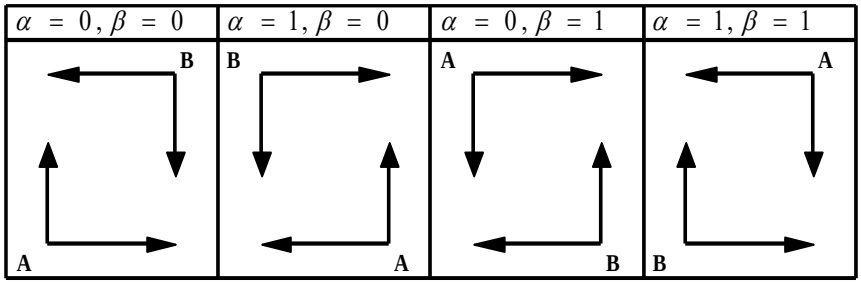
Если же оба направления оказываются заблокированными запрещёнными элементами, либо достигнут край координатной сетки, то движение луча прекращается.

Выбор направления движения лучей происходит исходя из интуиции разработчика . Существуют варианты распространения лучей, которые выбираются из таблицы, после вычисления значений α и β.

$$\alpha = \begin{cases} 1, если & X_A - X_B \geq 0 \\ 0, если & X_A - X_B < 0 \end{cases} \qquad \beta = \begin{cases} 1, если & Y_A - Y_B \geq 0 \\ 0, если & Y_A - Y_B < 0 \end{cases}$$

где (X_A, Y_A) – координаты начального элемента,
(X_B, Y_B) – координаты конечного элемента.

Исходя из значений α и β выбирается вид распространения лучей:



Пример 2. *Осуществить трассировку элементов A и B .*

10										
9			A	1	2	3	4	5	6	7
8			1							6
7			2							5
6			3				7	6		4
5			4					5		3
4			5	6	7			4		2
3								3		1
2								2	1	B
1										
	1	2	3	4	5	6	7	8	9	10

$\alpha = 3 - 10 = -7$

$\beta = 9 - 2 = 7$

$\alpha = 0$

$\beta = 1$

A

B

- Вариант распространения лучей

52 Четырехлучевой алгоритм

Из начальной и конечной точек выходят одновременно по четыре луча. Лучи движутся строго по заданным направлениям и “затухают”, если достигли края координатной сетки, либо встретили запрещенный элемент.

	1	
1	A	1
	1	

	1	
1	B	1
	1	

Пример 3.

	2					
	1					
1	A					
	1			1		
	2	2	1	B	1	2

53 Маршрутный алгоритм.

Маршрутный алгоритм получил свое название, потому что осуществляет одновременно и формирование фронта и прокладывание трассы. Источником волны на каждом шаге является конечный элемент участка трассы проложенной на предыдущих шагах.

В маршрутном алгоритме рассматриваются восьмиэлементная окрестность исходного элемента.

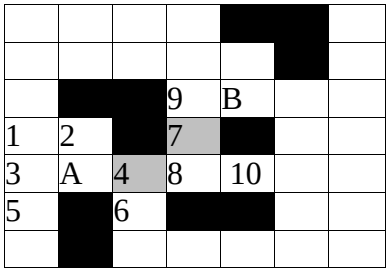
i-1,j-1	i,j-1	i+1,j-1
i-1,j	A	i+1,j
i-1,j+1	i,j+1	i+1,j+1

От каждого элемента окружения оценивается расстояние d до конечного элемента B.

$$d = \sqrt{(x_i - x_b)^2 + (y_i - y_b)^2} \text{ или } d = |x_i - x_b| + |y_i - y_b|$$

Таким образом определяются восемь значений расстояний, из которых выбирается минимальное. Элемент для которого d оказалось минимальным считаем элементом трассы. Процесс повторятся до тех пор пока расстояние не будет равным нулю (d=0) т.е. пока не будет достигнут конечный элемент. Обход запрещенных элементов осуществляется исходя из интуиции разработчика.

Пример:



$$d1 = \sqrt{(1-5)^2 + (4-5)^2} = \sqrt{17}$$

$$d2 = \sqrt{(2-5)^2 + (4-5)^2} = \sqrt{10}$$

$$d3 = \sqrt{(1-5)^2 + (3-5)^2} = \sqrt{20}$$

$$d4 = \sqrt{(3-5)^2 + (3-5)^2} = \sqrt{8}$$

$$d5 = \sqrt{(1-5)^2 + (2-5)^2} = \sqrt{25} = 5$$

$$d6 = \sqrt{(3-5)^2 + (2-5)^2} = \sqrt{13}$$

$$min = d4;$$

$$d7 = \sqrt{2};\ d8 = \sqrt{5};$$

$$d9 = 1;\ d10 = 2;$$

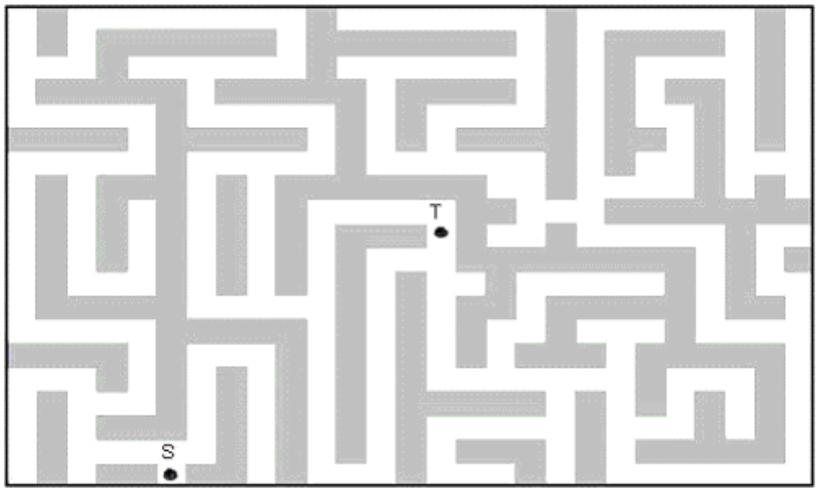
$$min = d7;$$

$$de = 0;$$

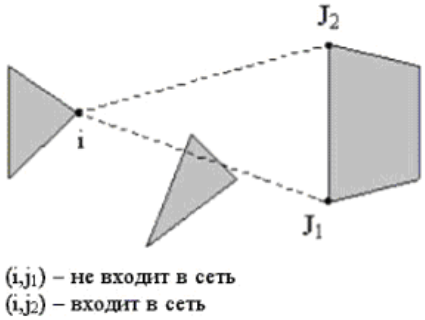
54 *Геометрическая модель задачи о лабиринте*

В лабиринте с произвольными препятствиями найти кратчайший путь между заданными точками.

Решение: Так как препятствия на местности образуют многоугольники, или какие либо другие геометрические фигуры (которые с некоторыми погрешностями тоже можно изобразить в виде многоугольников), то кратчайшая трасса будет являться ломанной с узлами в вершинах этих многоугольников. Звено ломаной – это либо сторона многоугольника, либо прямолинейный отрезок, проходящий вне многоугольников и соединяющий две вершины одного и того же или разных многоугольников. Для решения этой задачи нужно построить сеть (ломаную), а так же соединить точки s и t с простреливаемыми из них вершинами, если эти точки не являются вершинами многоугольников.

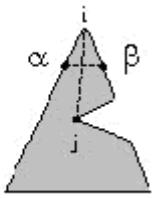


Формирование сети, т. е. матрицы расстояний C размером nxn (n – общее число вершин всех многоугольников плюс два для учета старта и финиша) представляет собой тройной цикл. Внешний – по i – перебор вершин, откуда стреляют; средний – по j (j от i+1 до n, чтобы не повторяться) – это перебор вершин, куда стреляют; и внутренний – по k – это проверка, не пересекает ли k-я сторона какого-либо многоугольника отрезок соединения.



Последнее условие, в случае, как на рис. 8,проверяется по стандартным формулам аналитической геометрии: выписывается уравнение прямой, проходящей через i, j, выписывается уравнение прямой проходящей через концы отрезка k, решением системы из этих двух уравнений находится точка пересечения и устанавливается, лежит ли точка

пересечения внутри рассматриваемых отрезков. Если да, то $d_{ij}=\infty$, конец цикла по k, если нет пересечения по окончанию цикла по k, то вычисляется Евклидово расстояние d_{ij}.



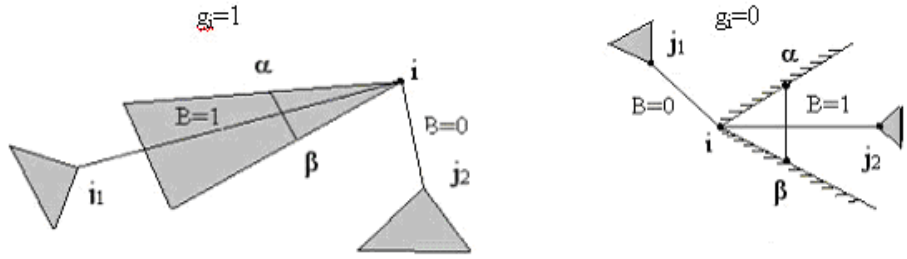
В случае на рис. 9, ситуация сложнее: между вершинами i и j не проходит ни какой стены, а j из i не простреливается. Чтобы преодолеть эту трудность, нужно ввести характеристику i угла препятствия: g_i присвоив g_i =0, если $\alpha_i<\pi$(“вогнутый” угол), или gi =1, если $\alpha_i>\pi$(“выпуклый” угол). Так, для угла с вершиной i на рис. 9 g_i =1, а для угла с вершиной j g_i =0.

Если крайние вершины x_i и x_{i+3} (x_i, x_{i+1}, x_{i+2}, x_{i+3} – последовательные вершины многоугольника) лежат по одну сторону от прямой, проходящей через соседние вершины x_{i+1}, x_{i+2}, то $g_{i+1}= g_{i+2}$,иначе $g_{i+1}<> g_{i+2}$.

(x- xi+1)(yi+2-yi+1)-(xi+2-xi+1)(y- yi+1)=0

Если при подстановке в это уравнение точек (xi, yi) и (xi+3, yi+3) в левой части получаются числа с одинаковым знаком, то $g_{i+1}= g_{i+2}$, иначе $g_{i+1}<> g_{i+2}$. После этого цикла будут известны все g_i точно или с точностью до наоборот. Остается абсолютно установить g_i хотя бы для одной вершины. Это легко сделать, потому что экстремальная вершина имеет g_0 =1.

Теперь можно справиться с трудностью, показанной на рис. 9. Из вершины i не простреливается никакая вершина j, защищенная углом с вершиной i. Чтобы исключить из рассмотрения загороженные вершины, нужно отступить от вершины i по сторонам угла на величину ε,заведомо меньшую, чем длина стороны, построив таким образом точки αи β. После чего нужно ввести бинарную величину B, B=1, если отрезки $\alpha\beta$и ij пересекаются; и B=0, если отрезки $\alpha\beta$и ij не пересекаются. Как например на рис. 10



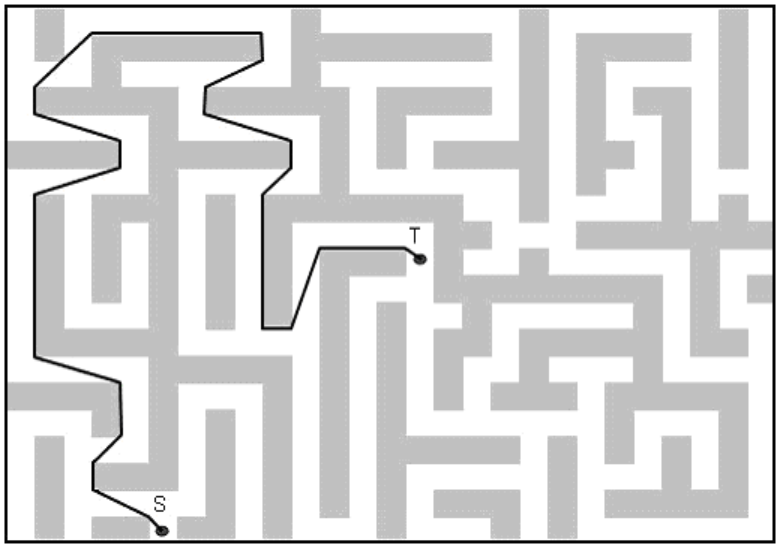
Всего имеется четыре возможности:

1) B=1 и gi=0
2) B=0 и gi=1
3) B=1 и gi=0
4) B=1 и gi=1

Ясно что вершина j не простреливается – в случаях 2 и 3 (при нечетном B+g).

Теперь можно построить сеть.

После того как сеть построена, можно приступать к нахождению кратчайших путей, воспользовавшись любым из выше рассмотренных алгоритмов (в зависимости от поставленной задачи).



55 *Алгоритмы составления расписания.*

Предположим, что имеется множество *n* одинаковых процессоров, обозначенных $P_1, P_2, \ldots, P_n$, и m независимых заданий $J_1, J_2, \ldots, J_m$, которые нужно выполнить. Процессоры могут работать одновременно, и любое задание можно выполнять на любом процессоре. Если задание загружено в процессор, оно остается там до конца обработки. Время обработки задания J_i известно и равно $t_i, i = 1,2,\ldots,m.$ Организовать обработку заданий таким образом, чтобы выполнение всего набора заданий было завершено как можно быстрее. Система работает следующим образом: первый освободившийся процессор берет из списка следующее задание. Если одновременно освобождаются два или более процессоров, то выполнять очередное задание из списка будет процессор с наименьшим номером.

Пример. Пусть имеется три процессора и шесть заданий , время выполнения каждого из которых равно: $t_1 = 2, t_2 = 5, t_3 = 8, t_4 = 1, t_5 = 5, t_6 = 1.$

Рассмотрим расписание $L = (J_2, J_5, J_1, J_4, J_6, J_3).$ В начальный момент времени *T=0,* процессор P_1 начинает обработку задания J_2, процессор P_2 - задания J_5, а процессор P_3 - задания J_1. Процессор P_3 заканчивает выполнение задания J_1 в момент времени $T = 2$ и начинает обрабатывать задание J_4, пока процессоры P_1 и P_2 все еще работают над своими первоначальными заданиями. При *T=3* процессор P_3 опять заканчивает задание J_4 и

начинает обрабатывать задание J_6, которое завершается в момент *T=4*. Тогда он начинает выполнять последнее задание J_3. Процессоры P_1 и P_2 заканчивают задания при *T=5*, но, так как список *L* пуст, они останавливаются. Процессор P_3 завершает выполнение задания J_3 при *T=12*. Рассмотренное расписание проиллюстрировано на рис.1. временной диаграммой, известной как *схема Ганта*. Очевидно, что расписание не оптимально. Можно «подобрать», например, расписание $L^* = (J_3, J_2, J_5, J_1, J_4, J_6)$, которое позволяет завершить все задания за *T* = 8* единиц времени (рис.2.).

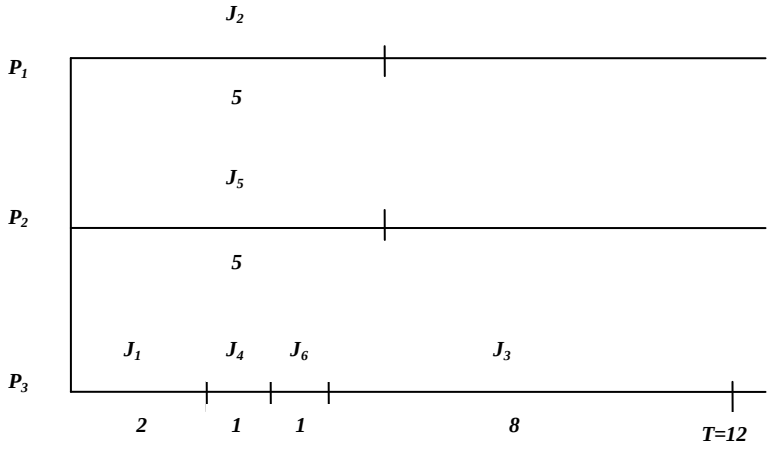


Рис.1. Схема Ганта для расписания L

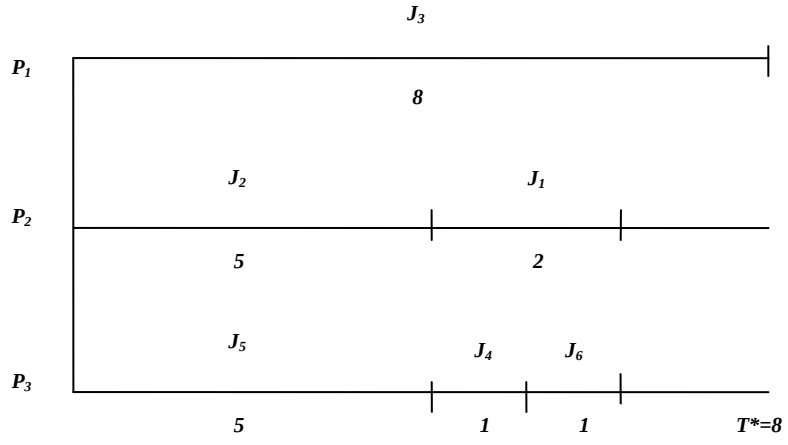


Рис.2. Схема Ганта для оптимального расписания

Теперь рассмотрим другой тип задач по составлению расписания для многопроцессорных систем. Вместо вопроса о быстрейшем завершении набора заданий фиксированным числом процессоров теперь поставим вопрос о минимальном числе

процессоров, необходимых для завершения данного набора заданий за фиксированное время T_o. Конечно, время T_o будет не меньше времени выполнения самого трудоемкого задания.

В такой постановке задача составления расписания эквивалентна следующей задаче упаковки. Пусть каждому процессору P_j соответствует ящик B_j размера T_o. Пусть каждому заданию J_i соответствует предмет размера t_i, равного времени выполнения задания J_i, где $i = 1,2,...,n.$ Теперь для решения задачи по составлению расписания нужно построить алгоритм, позволяющий разместить все предметы в минимальном количестве ящиков. Конечно, нельзя заполнять ящики сверх их объема T_o, и предметы нельзя дробить на части.

56 Литература

1. Т. Кормен, Ч. Лейзерсон, Р. Ривест
   Алгоритмы: построение и анализ. М.: МЦНМО, 2000.
2. Д.Кнут Искусство программирования , том 1. Основные алгоритмы. Уч . пос. М.:Изд. Дом " Вильямс ", 2000.
3. Вирт Н. Алгоритмы и структуры данных.: Пер. С англ. - М.: Мир, 2001.
4. Хусаинов Б.С. Структуры и алгоритмы обработки данных. Примеры на
   языке Си. Учеб. пособие. М : Финансы и статистика, 2004.
5. А. Ахо, Дж.Хопкрофт, Дж.Ульман, Структуры данных и алгоритмы М: СПб: Киев: Вильямс, 2001г.

57 Лабораторное задание.

Для выполнения лабораторной работы необходимо:

1) Ознакомиться с эвристическими алгоритмами.
2) Осуществить трассировку элементов интегральных схем, размером 10x10, 20x20, 30x30. Зафиксировать параметры трассировок всеми рассмотренными методами.

Системы работает в диалоговом режиме с использованием «меню». Вся необходимая поясняющая информация отображается во время работы системы на экране монитора.

3) Составить оптимальное расписание работы четырех процессоров, для которых известно $t_1, \ldots, t_{11}$.
4) Составить алгоритм оптимальной упаковки 12 предметов , размером от1 до 4 в ящики размером 6.
5) Составить программу эвристического алгоритма (по заданию преподавателя)

58 Требования к отчету

Отчет должен содержать:

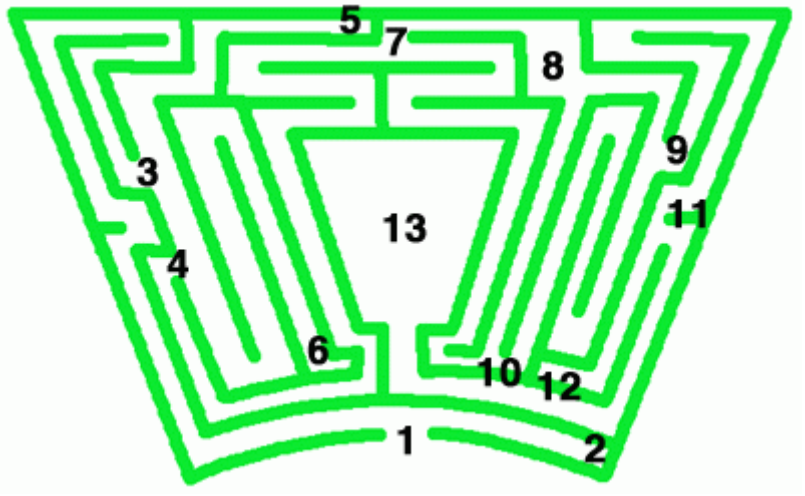
5)Конспект лабораторной работы;
6)Схемы волнового и лучевых алгоритмов;
7)Результаты выполнения работы;
8)Выводы по работе.

Контрольные вопросы

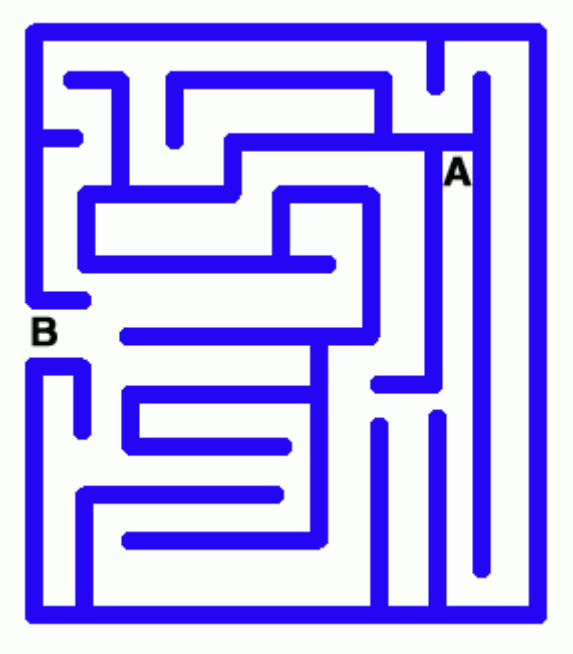
6) Какова теоретическая сложность алгоритмов, рассмотренных в данной работе?
7) Особенности работы волнового, лучевых и маршрутного алгоритмов?
8) Принципы составления оптимального расписания работы параллельных процессоров?
9) В чем особенности задачи упаковки?
10) Принципы решения задачи о джипе?
6) Как построить дерево решений в задаче о кодовом замке?

59 Решение задач

Пример 1. Найти выход из произвольной точки лабиринта в саду Хемптон Корт. Отождествив коридоры лабиринта с ребрами, а перекрестки, тупики, входы и выходы - с вершинами, перейти к связному графу, представляющему схему лабиринта.



Пример 2. Нарисуйте граф, соответствующий лабиринту. Найдите путь, по которому можно пройти от пункта А до В лабиринта, используя предложенные выше алгоритмы.



Пример 3. (Задача о джипе). Пусть необходимо пересечь на джипе ***1000*** -километровую пустыню, израсходовав при этом минимум горючего. Объем топливного бака джипа ***500*** литров, горючее расходуются равномерно, по одному литру на километр. При этом в точке старта имеется неограниченный резервуар с топливом. Так как в пустыне нет складов с горючим, необходимо установить собственные хранилища и наполнять их топливом из бака машины. Конечно, проще было бы ехать на грузовике, загруженным бочками с бензином, но тогда не было бы задачи о джипе.

Итак, идея задачи ясна: нужно из точки старта отъезжать с полным баком на некоторое расстояние, устраивать там первый склад, оставлять там какое-то количество горючего из бака, но такое, чтобы хватило вернуться назад. В точке старта вновь производится полная заправка и делается попытка второй склад продвинуть в пустыню дальше. Но где обустраивать эти склады и сколько горючего оставлять в каждом из них?

Пример 4. (Задача о кодовом замке). Пусть кодовый замок состоит из набора ***N*** переключателей, каждый из которых может быть в положении “вкл” или “выкл”. Замок открывается только при одном наборе положений переключателей, из которых не менее [***N/2***] (целая часть от ***N/2***) , находятся в положении “вкл”. Построить алгоритм перебора комбинаций, чтобы не пропустить нужную и не набирать ту, которая заведомо к успеху не приведет.

Промоделируем каждую возможную комбинацию вектором из ***N*** нулей и единиц. На ***i***-м месте будет ***1***, если ***i***-й переключатель находится в положении “вкл” и ***0***, если ***i***-й переключатель - в положении “выкл”. Множество всех возможных ***N***-векторов моделируется с помощью бинарного (или двоичного) дерева. Если количество переключателей в замке равно ***N***, то в дереве просмотра будет ***N*** уровней. Решить задачу для